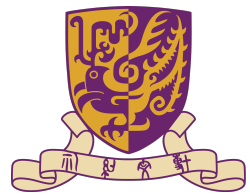
CSCI3230 (ESTR3108)
Fundamentals of Artificial Intelligence

Lecture 7. Convolutional Neural Networks

Prof. Qi Dou

Email: qidou@cuhk.edu.hk
Office: Room 911, 9/F, SHB

Dept. of Computer Science & Engineering
The Chinese University of Hong Kong



Outline

Part 1. Basic concepts

Part 2. CNN for classification

Part 3. Tricks for CNN training

Part 4. Summary

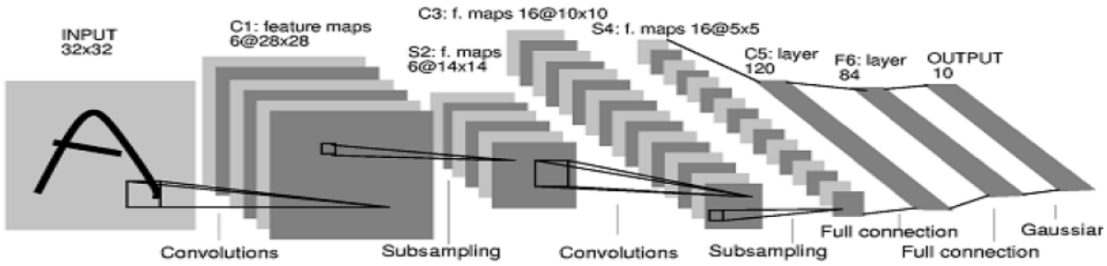
Part 1. Basic concepts

Two breakthrough architectures in CNN

LeNet

1998

LeCun et al.



# of transistors

10^6

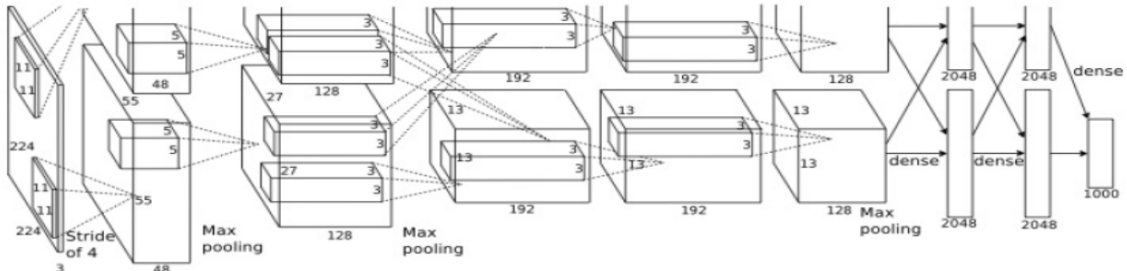
# of pixels used in training

10^7 NIST

AlexNet

2012

Krizhevsky et al.



# of transistors

10^9

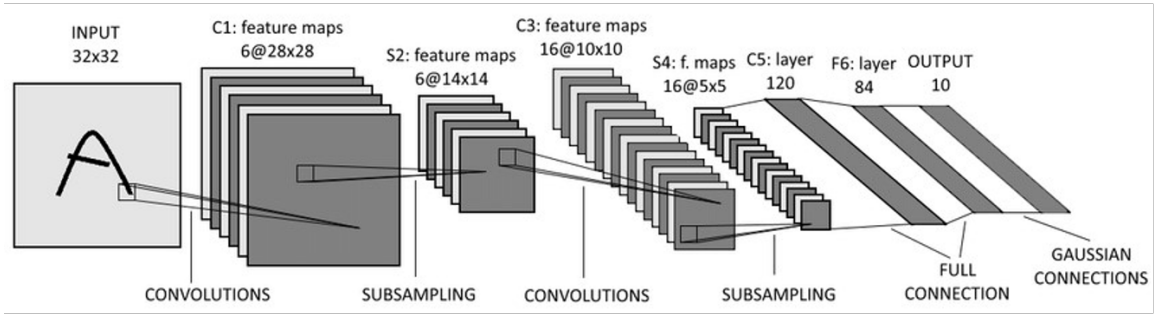
GPUs

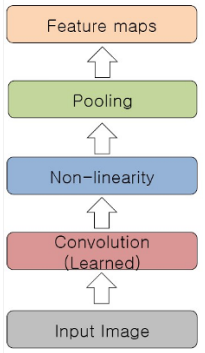
# of pixels used in training

10^{14} IMAGENET

LeNet

- LeNet-5[1] by LeCun et al. 1998
  - Convolve input
  - Non-linearity: ReLU
  - Pooling
- Good at MNIST/CIFAR-10/Traffic sign recognition.
- Less good at more complex datasets, e.g. Caltech-101/256

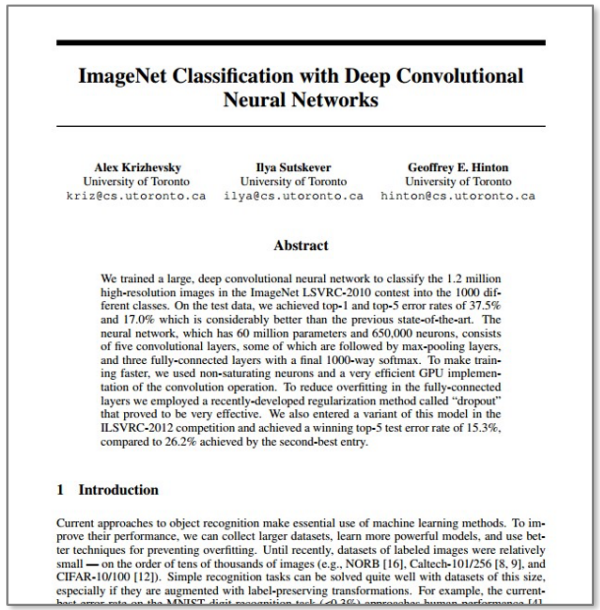




[1] http://yann.lecun.com/exdb/lenet/index.html

AlexNet

- AlexNet by Alex Krizhevsky et al. 2012
- ILSVRC-2012 Image recognition top-1 and top-5 error rates of 37.5% and 17.0%



ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky University of Toronto kriz@cs.utoronto.ca
Ilya Sutskever University of Toronto ilya@cs.utoronto.ca
Geoffrey E. Hinton University of Toronto hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

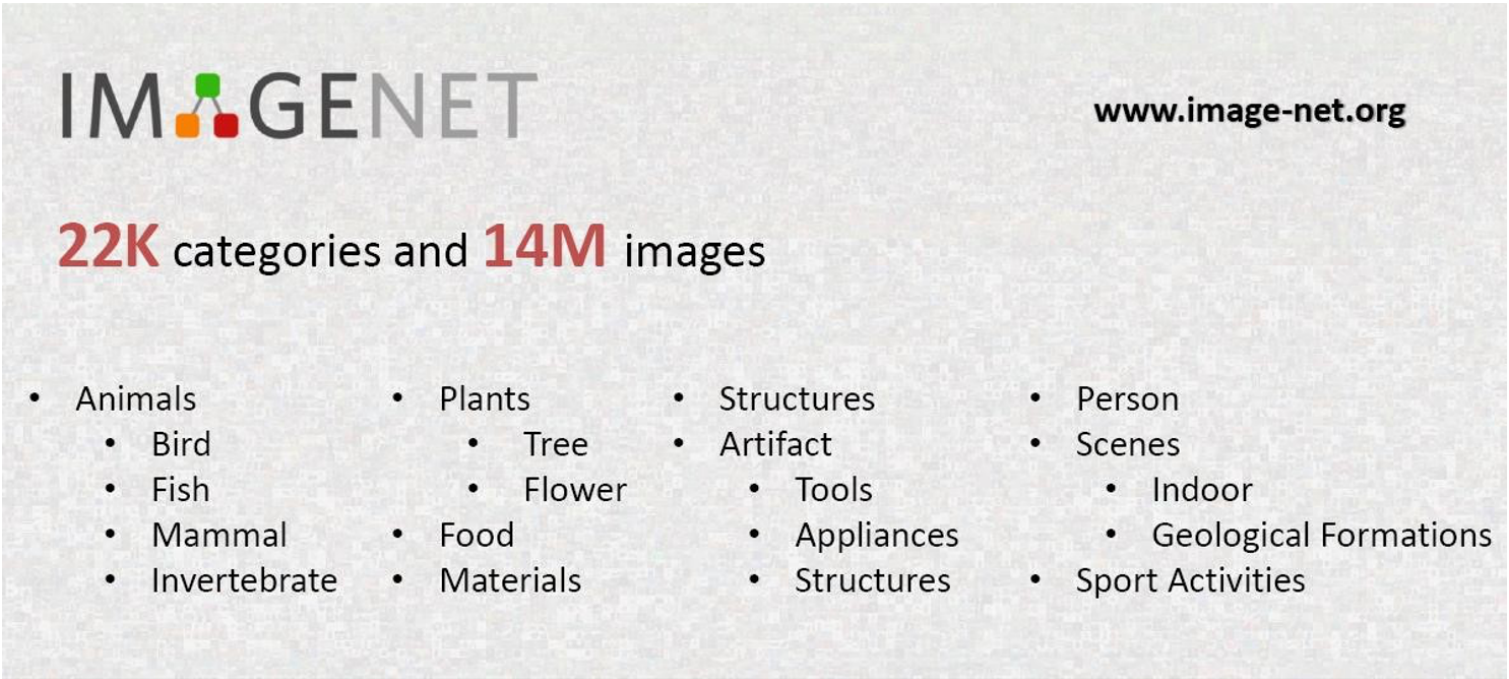
1 Introduction

Current approaches to object recognition make essential use of machine learning methods. To improve their performance, we can collect larger datasets, learn more powerful models, and use better techniques for preventing overfitting. Until recently, datasets of labeled images were relatively small — on the order of tens of thousands of images (e.g., NORB [16], Caltech-101/256 [8, 9], and CIFAR-10/100 [12]). Simple recognition tasks can be solved quite well with datasets of this size, especially if they are augmented with label-preserving transformations. For example, the current-

Krizhevsky, A., Sutskever, I., & Hinton, G. E. Imagenet classification with deep convolutional neural networks. NeurIPS, 2012.

ImageNet

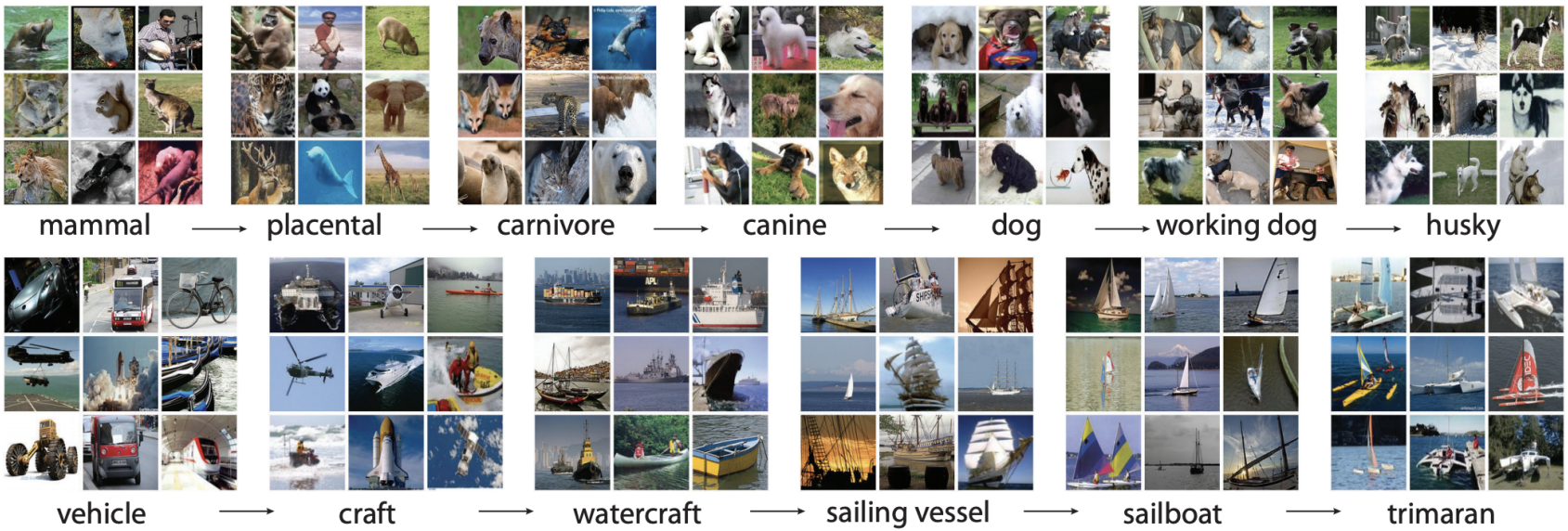
- ImageNet for Large Scale Visual Recognition Challenge (ILSVRC)



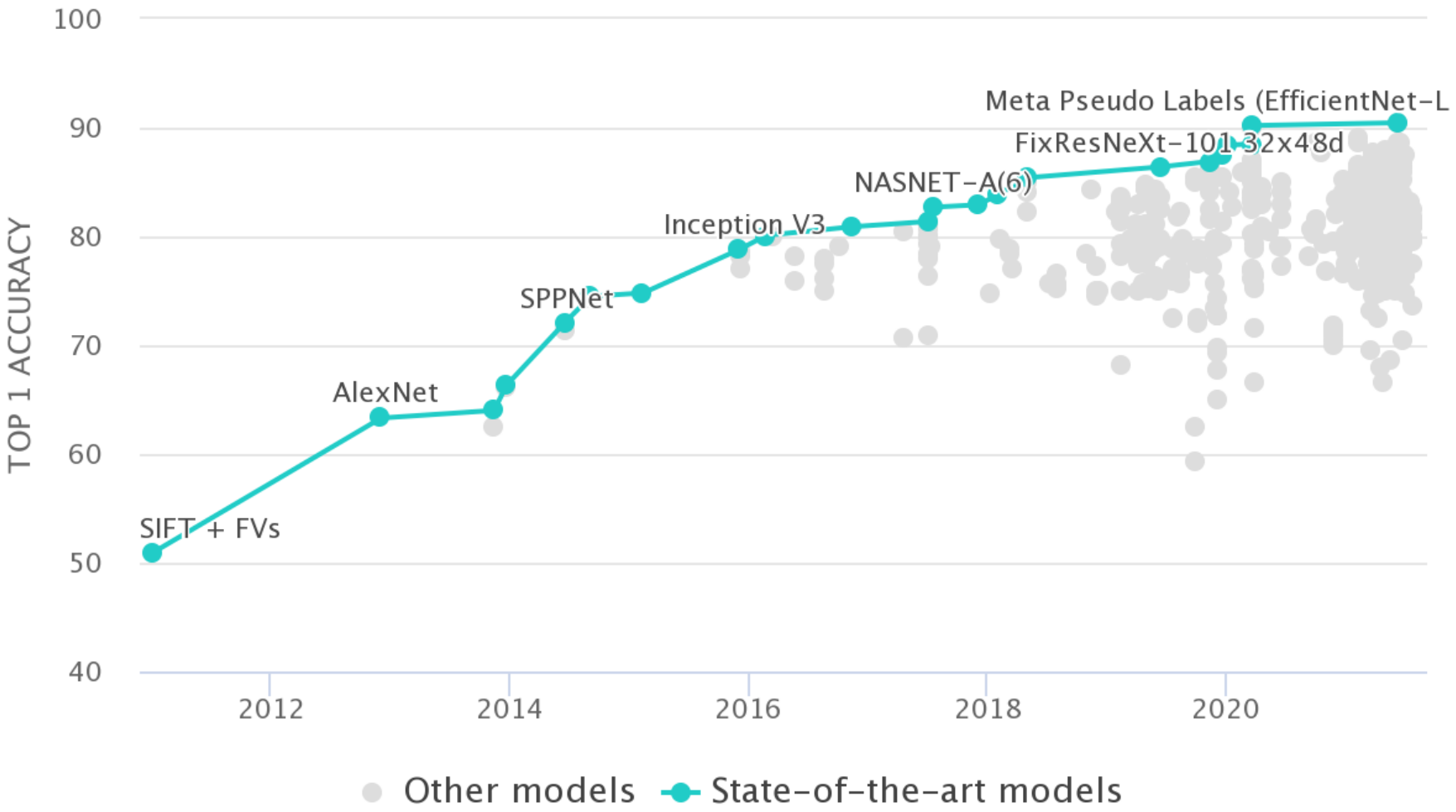
http://www.image-net.org/challenges/LSVRC/

ImageNet

- ImageNet dataset overview

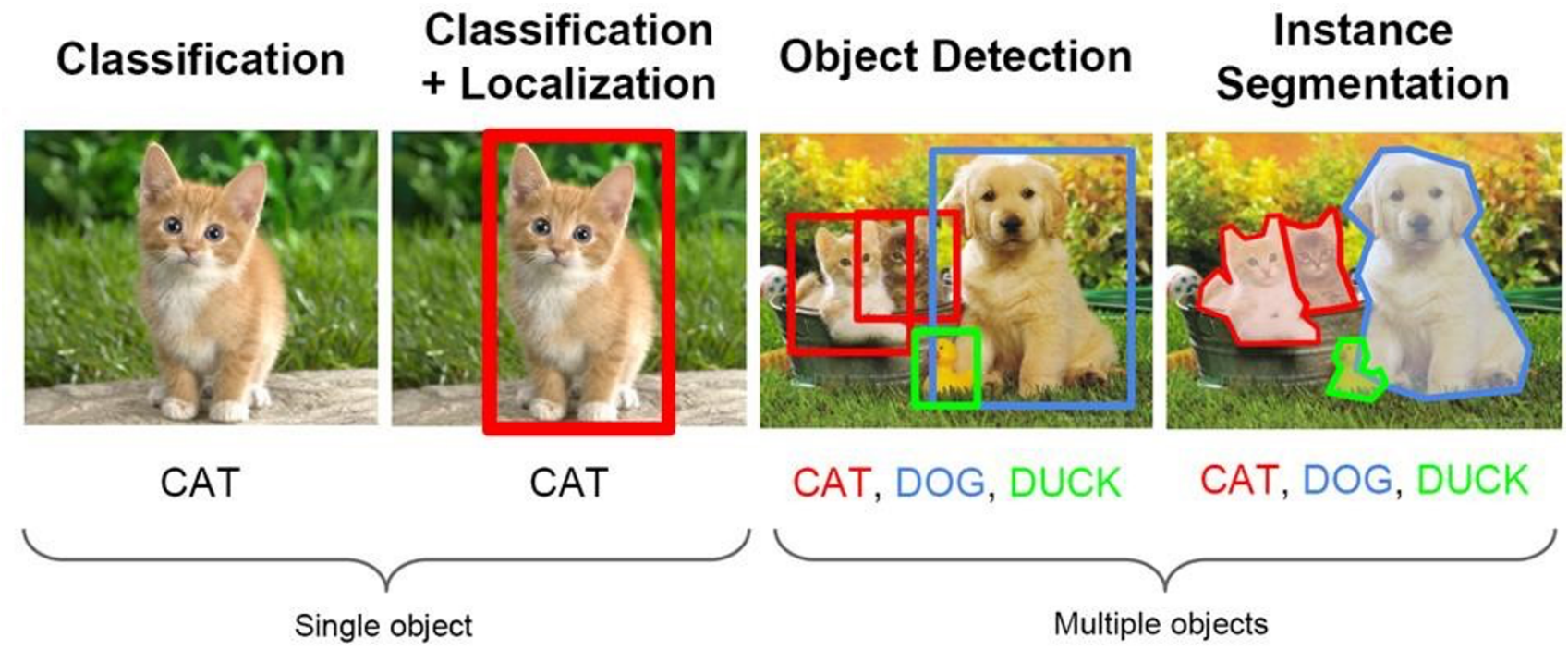


ImageNet classification (2010-2021)

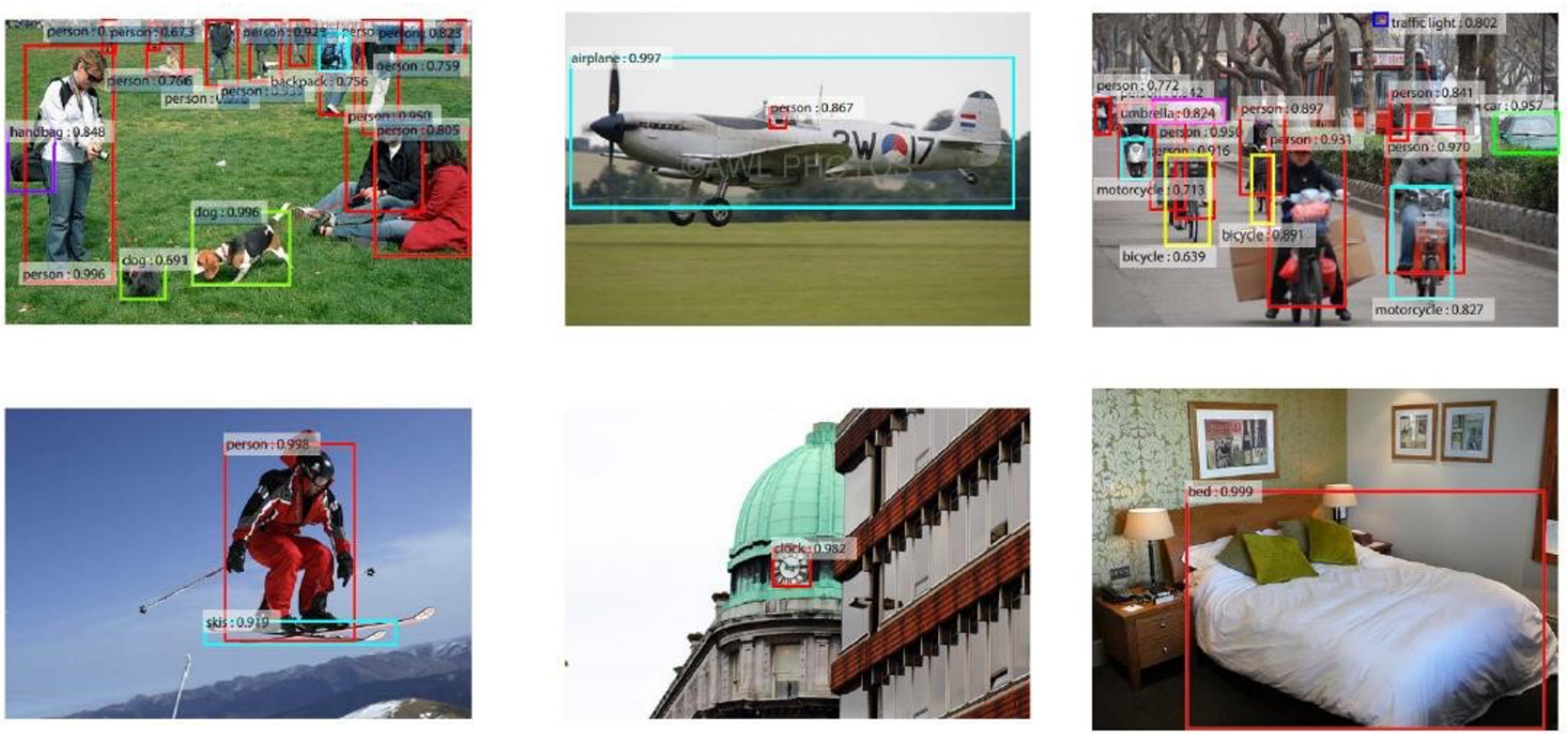


https://www.paperswithcode.com/sota/image-classification-on-imagenet

Computer vision tasks



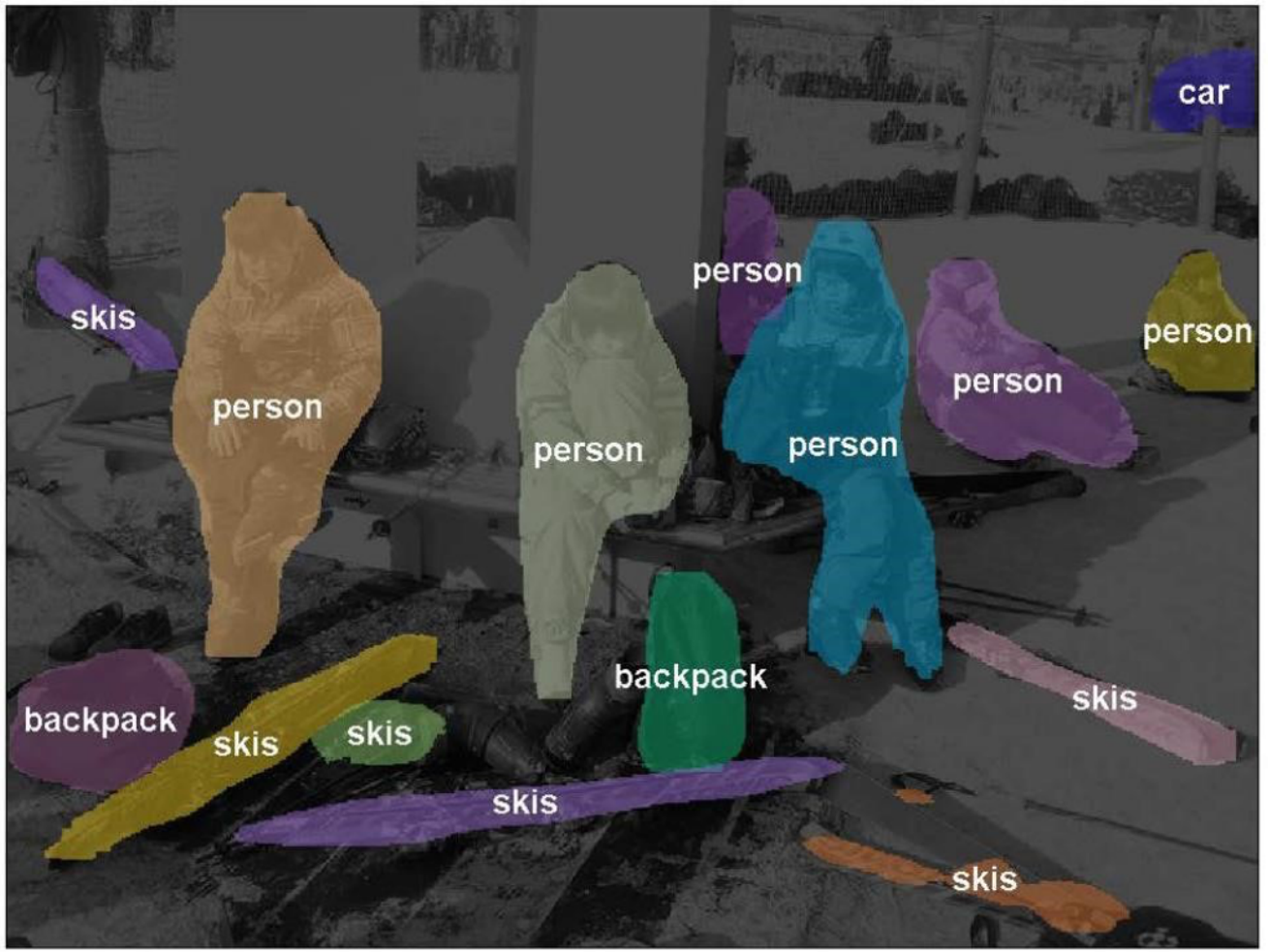
Localization and detection



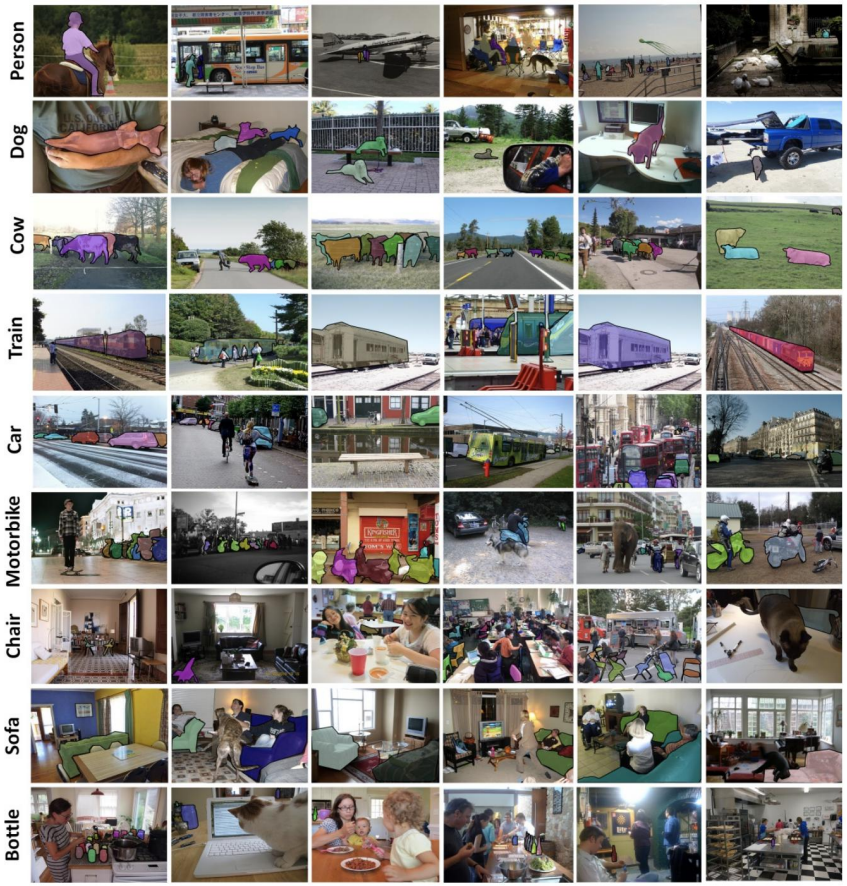
Results from Faster R-CNN, Ren et al 2015

Instance segmentation

input

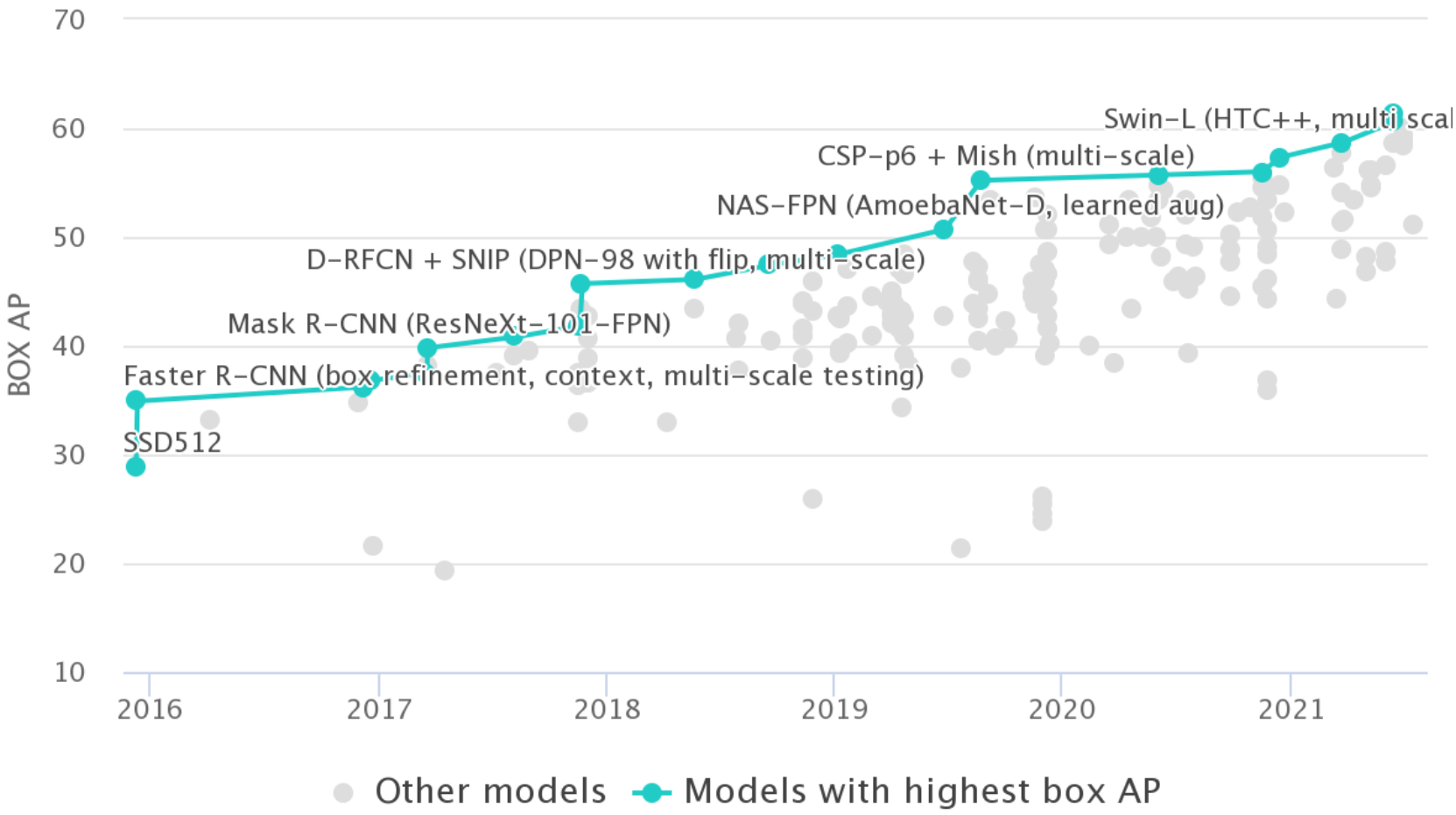


Microsoft COCO dataset



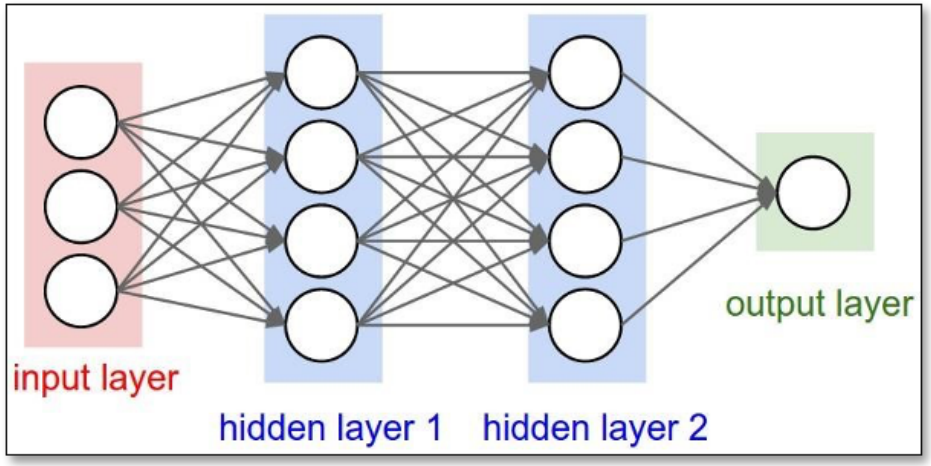
Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C.L., Microsoft coco: Common objects in context. In European conference on computer vision (ECCV), 2014.

COCO object detection (2016-2021)



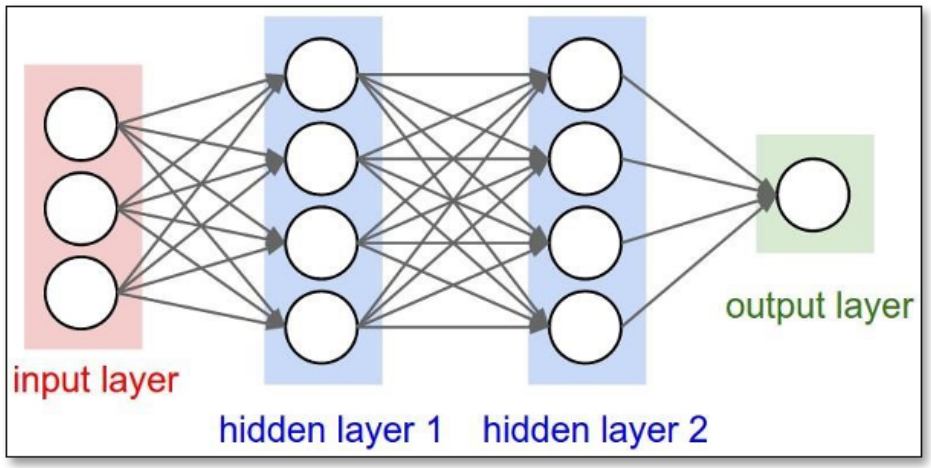
From NN to CNN

- A regular 3-layer Neural Network.

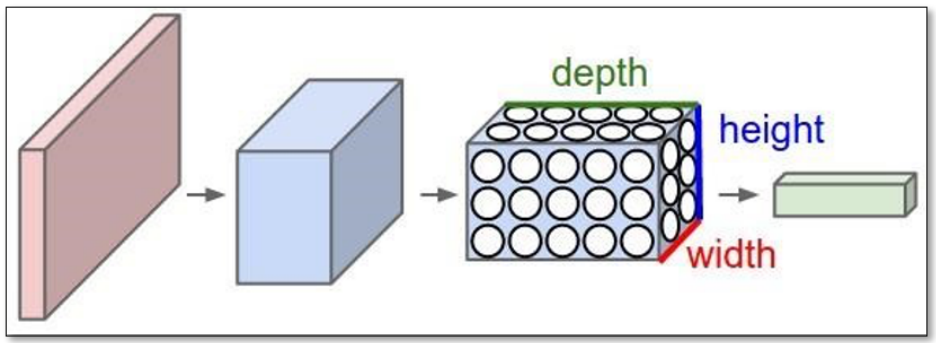


From NN to CNN

- A regular 3-layer Neural Network.

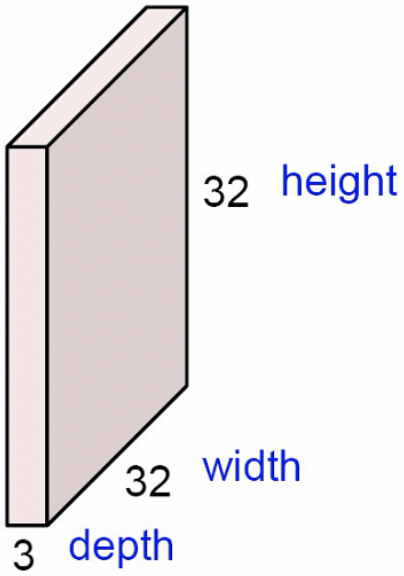


- A Convolutional Neural Network (CNN or ConvNet) arranges its neurons in three dimensions, i.e., 3D output tensor.

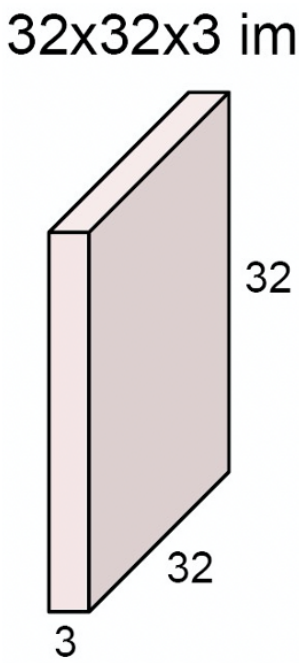


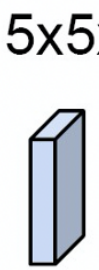
Convolutional layer

32x32x3 image -> preserve spatial structure



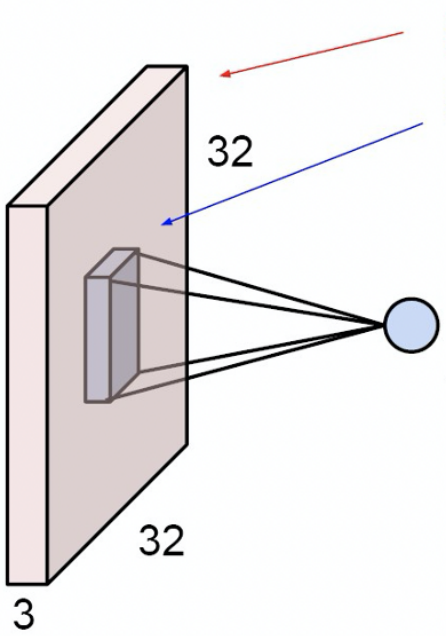
Convolutional layer





Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

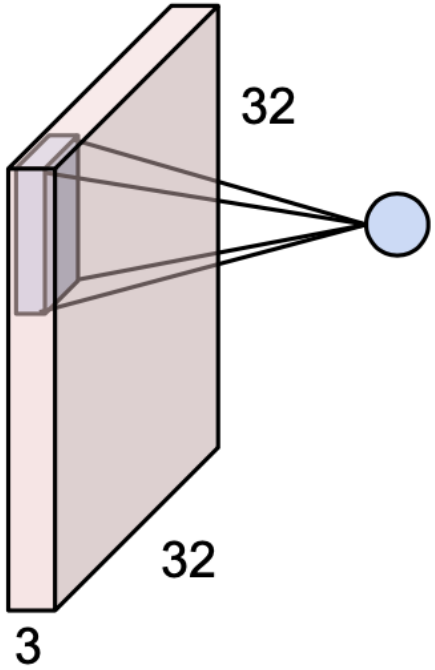
Convolutional layer



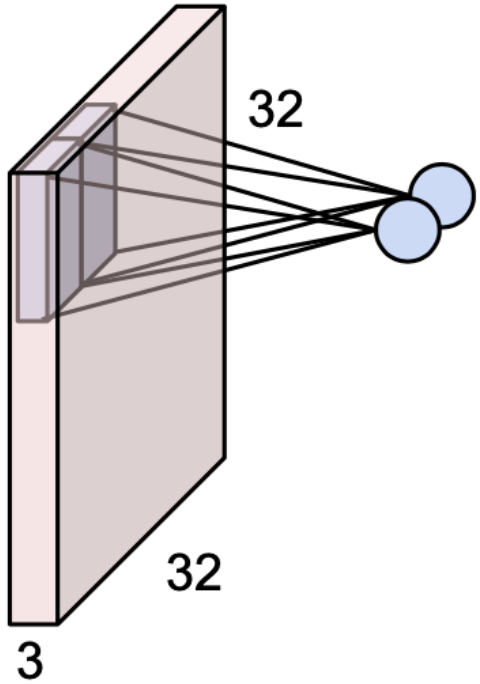
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

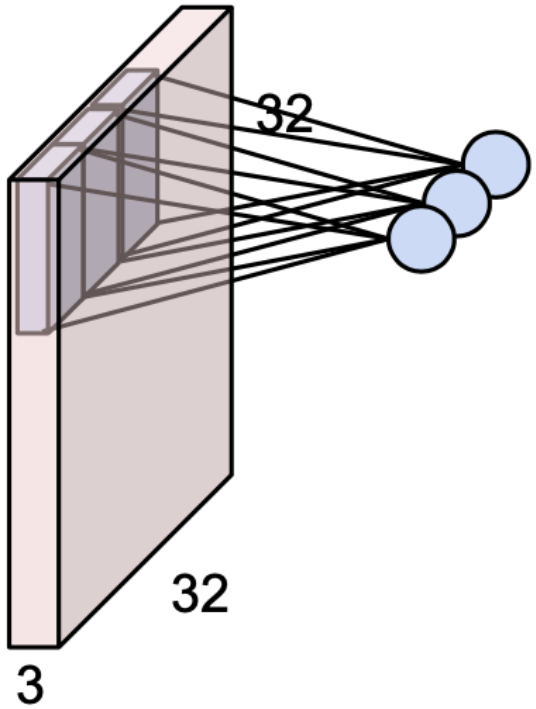
Convolutional layer



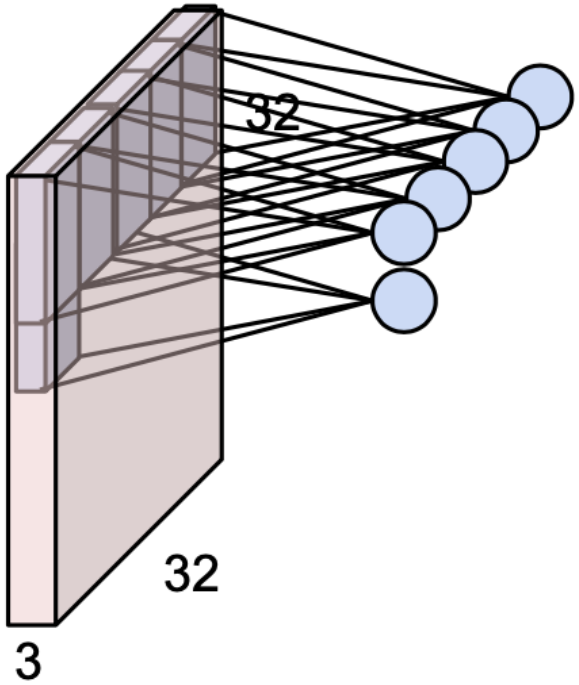
Convolutional layer



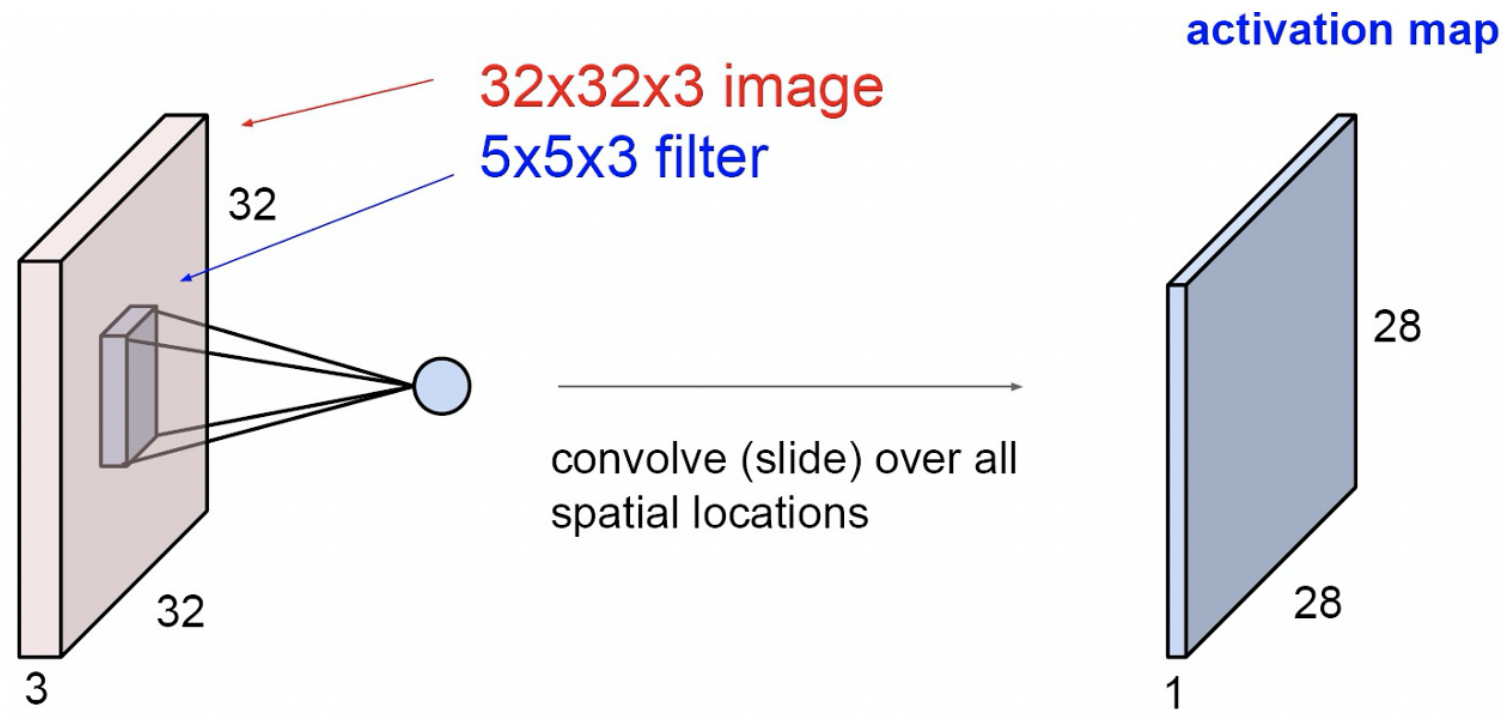
Convolutional layer



Convolutional layer

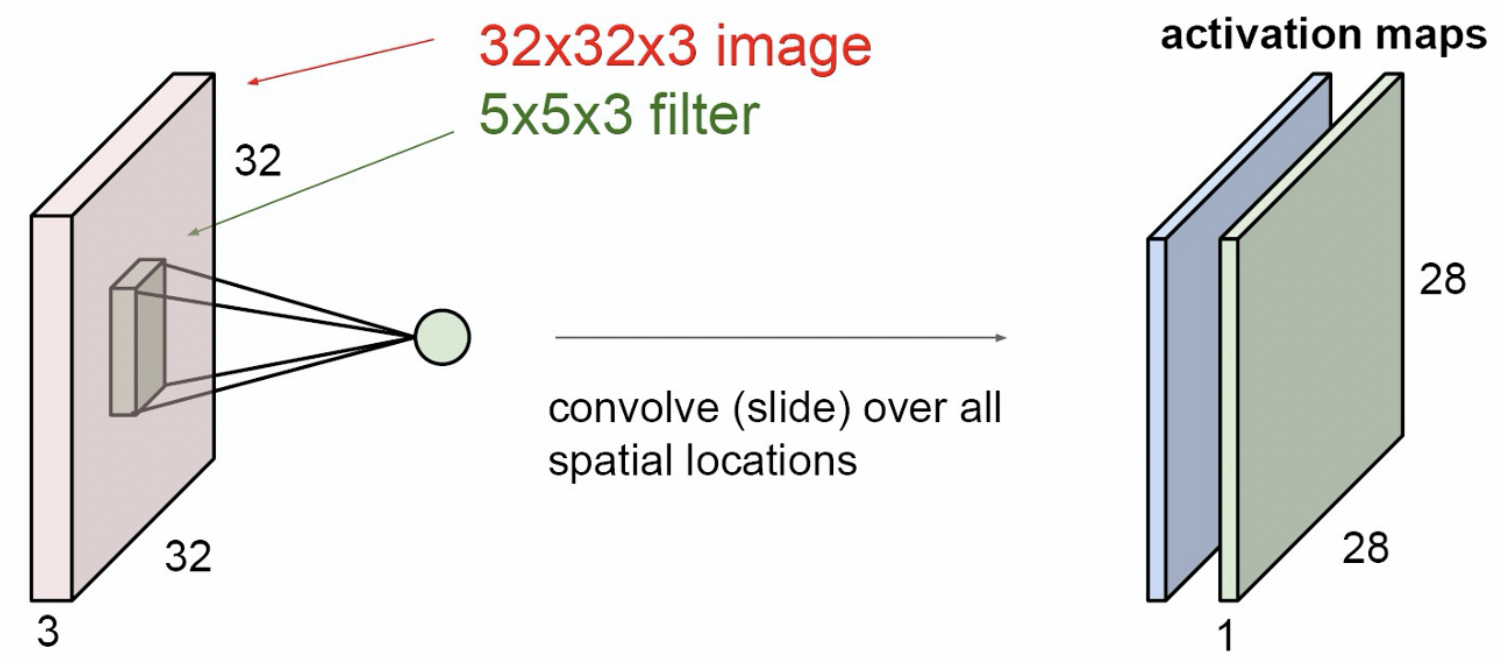


Convolutional layer



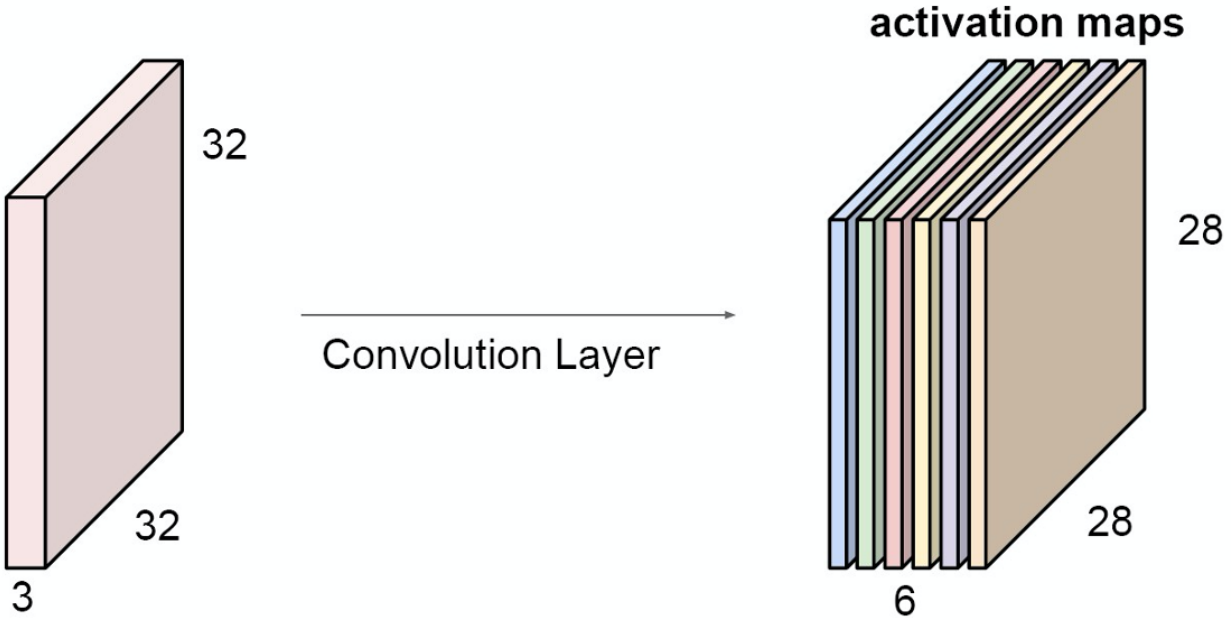
Activation map is also called feature map.

Convolutional layer



Convolutional layer

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

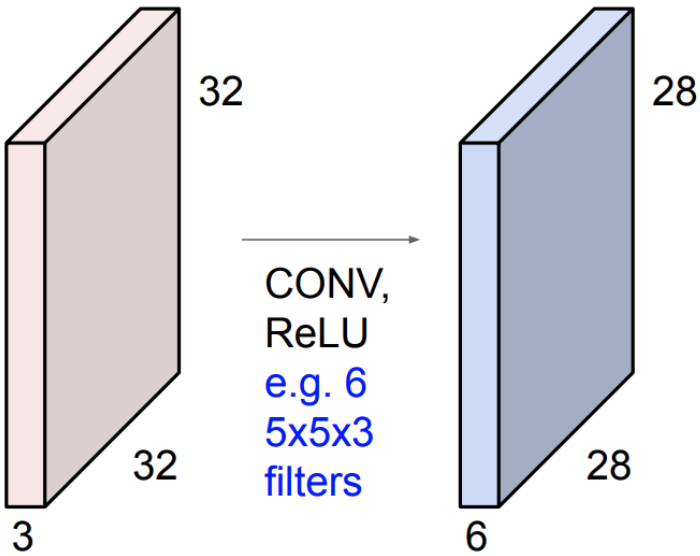


We stack these up to get a "new image" of size 28x28x6!

The output dimension is also known as channel.

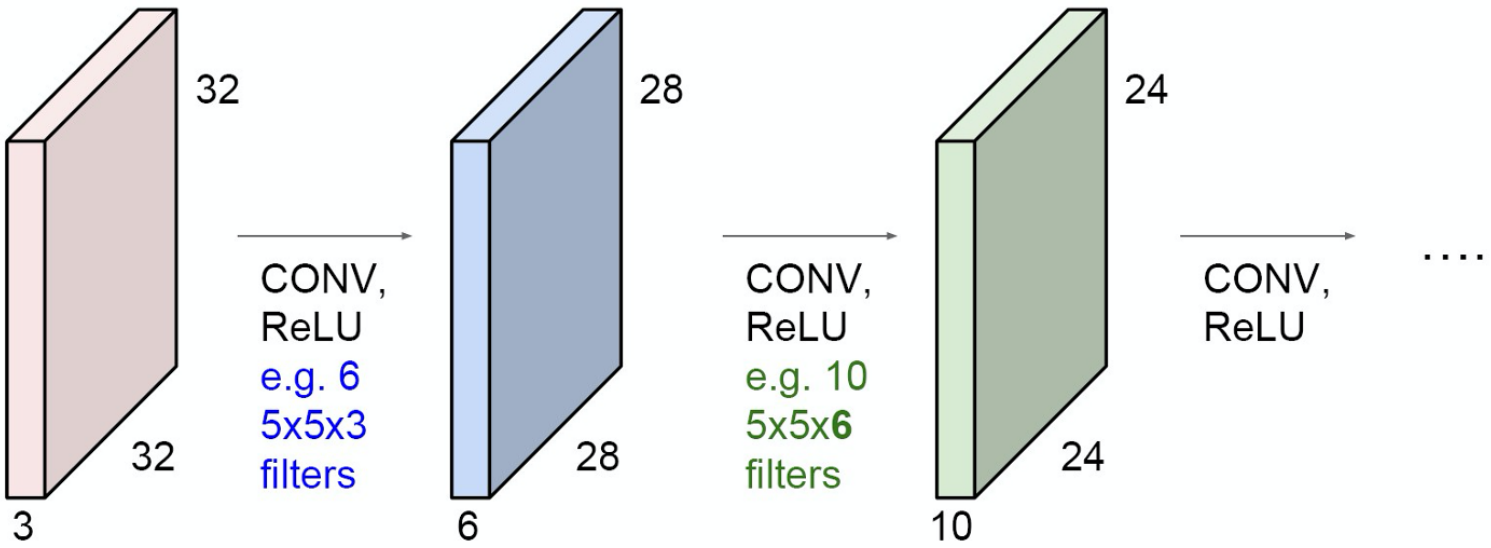
Convolutional layer

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions

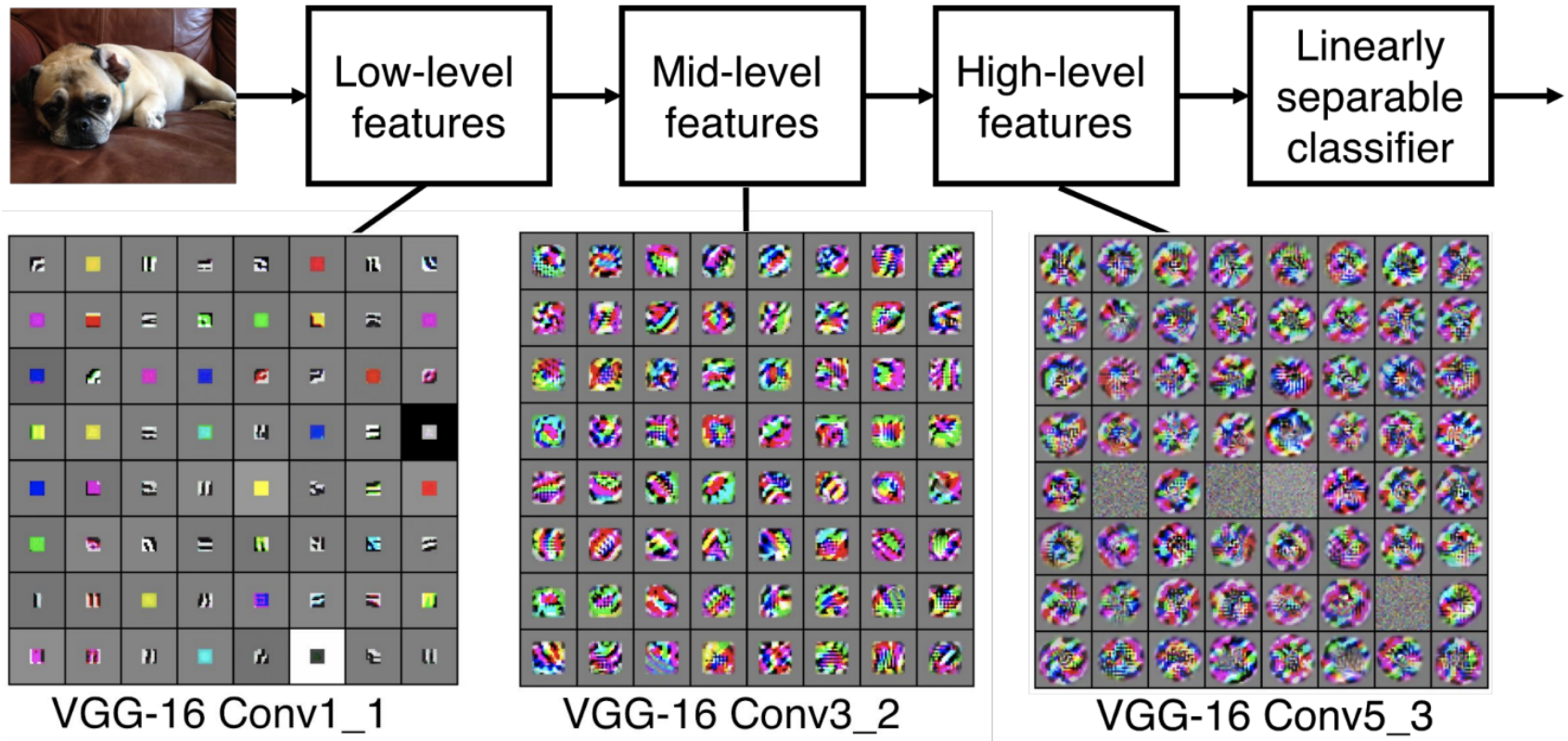


Convolutional layer

Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



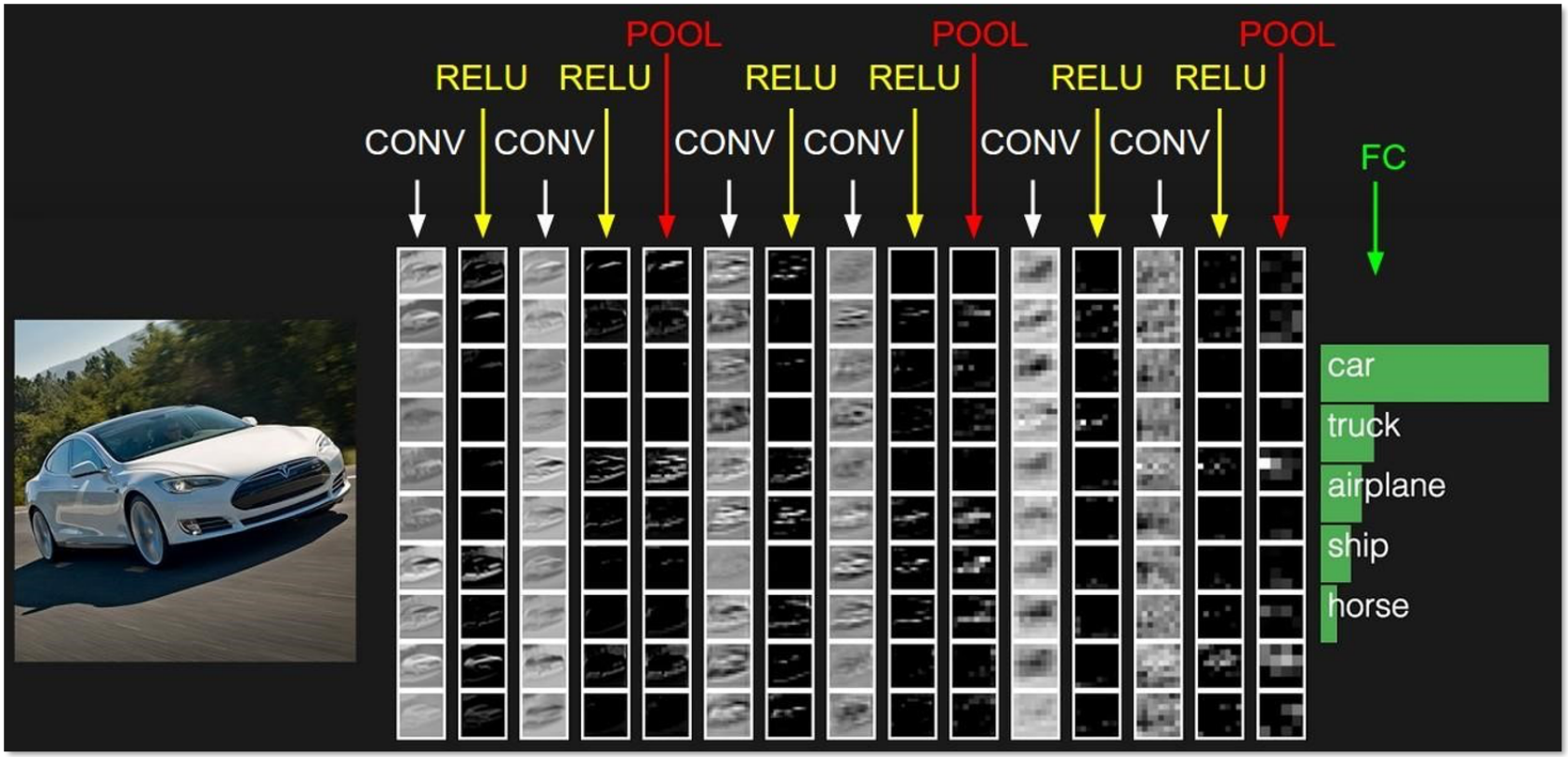
Convolutional layer



Zeiler, M.D. and Fergus, R., Visualizing and understanding convolutional networks. In European conference on computer vision (ECCV), 2014

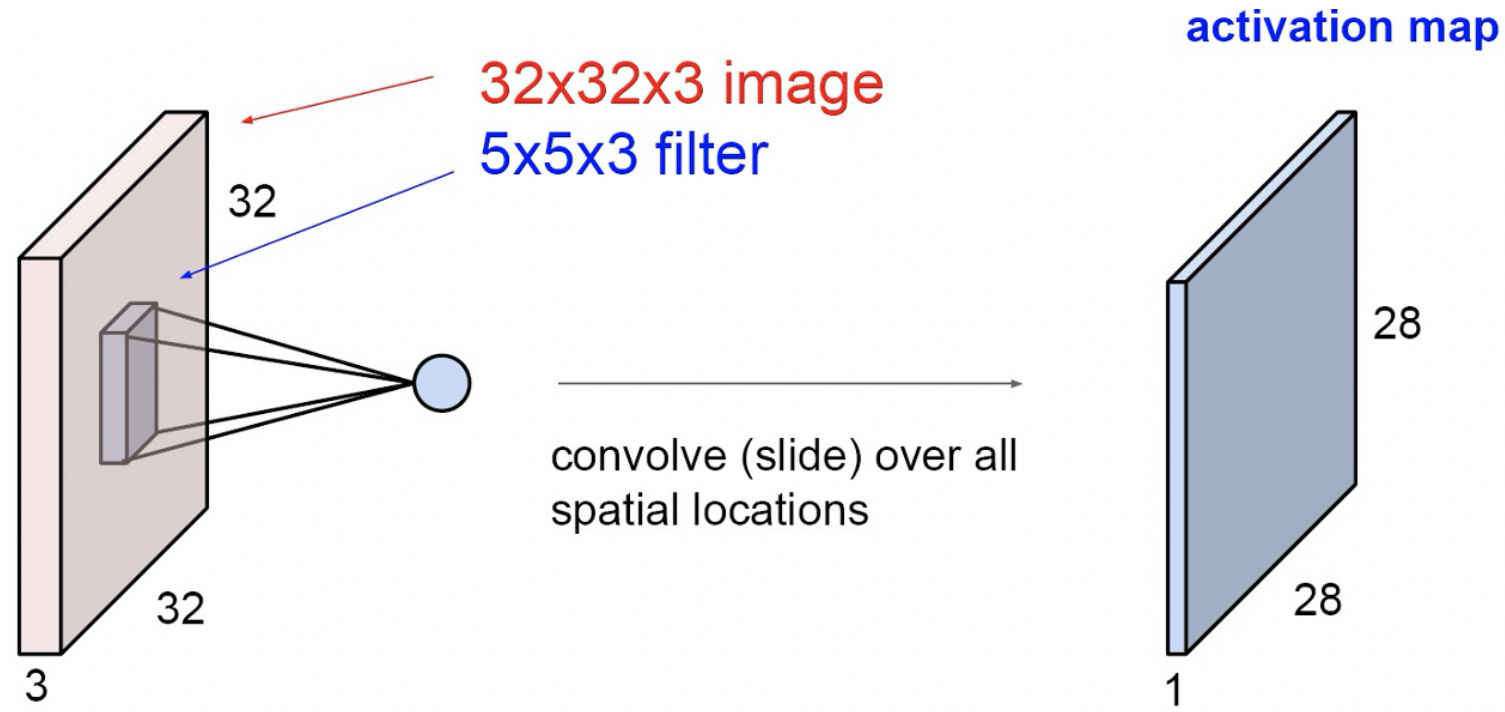
Convolutional layer stack

- Example of a CNN with hierarchical architecture



Convolutional layer spatial dimension analysis

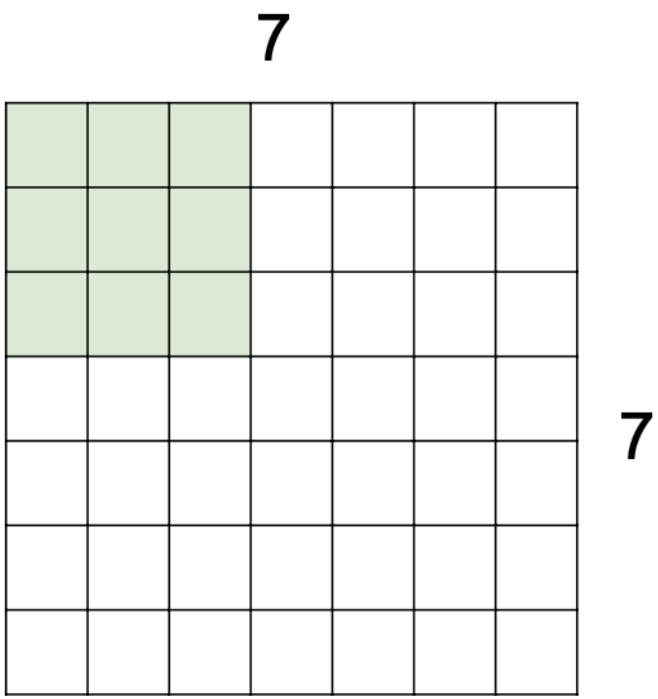
- A closer look at spatial dimensions



Convolutional layer

- A closer look at spatial dimensions

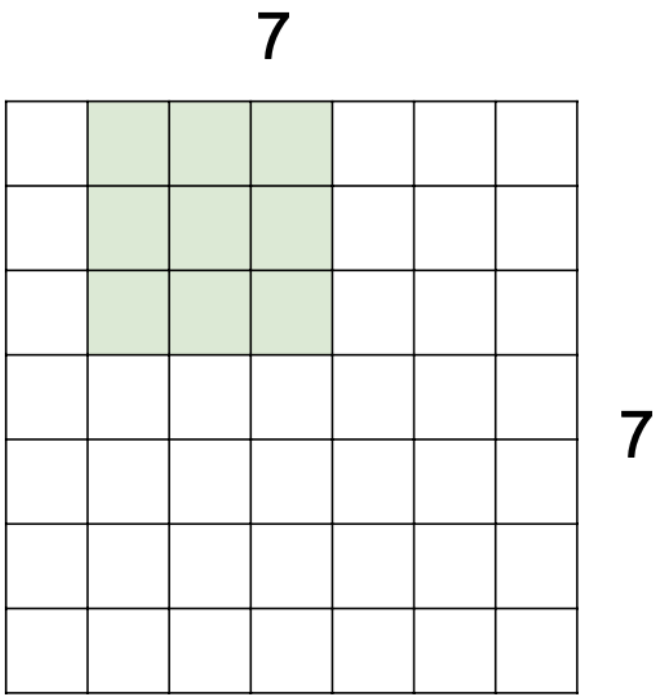
7×7 input
assume 3×3 filter



Convolutional layer spatial dimension analysis

- A closer look at spatial dimensions

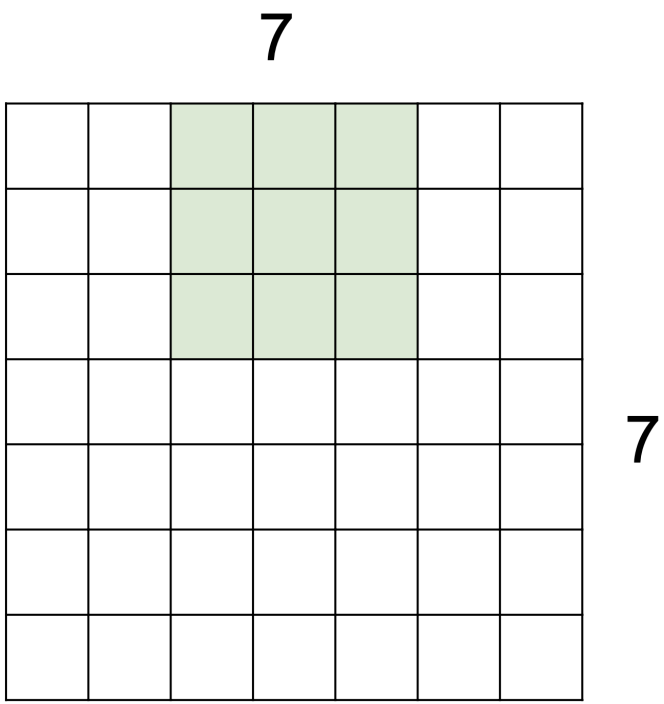
7×7 input
assume 3×3 filter



Convolutional layer spatial dimension analysis

- In practice: common to zero pad the border.

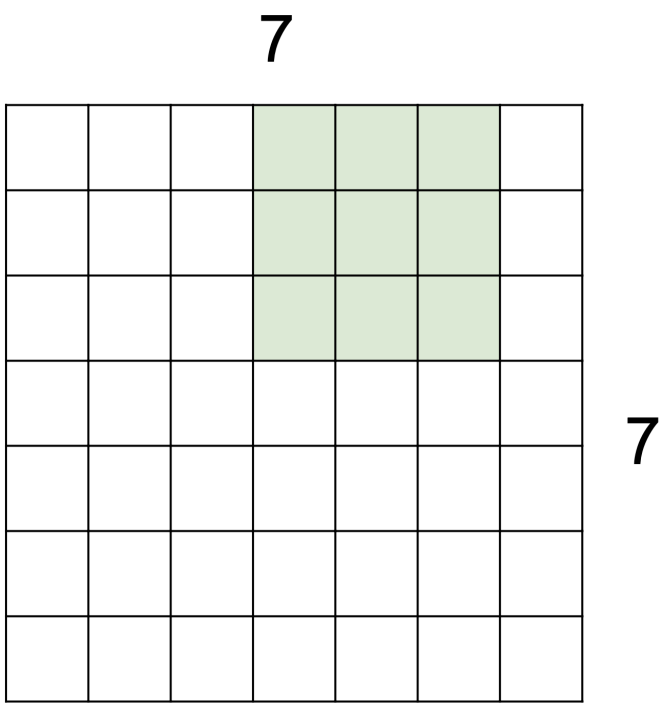
7×7 input
assume 3×3 filter



Convolutional layer spatial dimension analysis

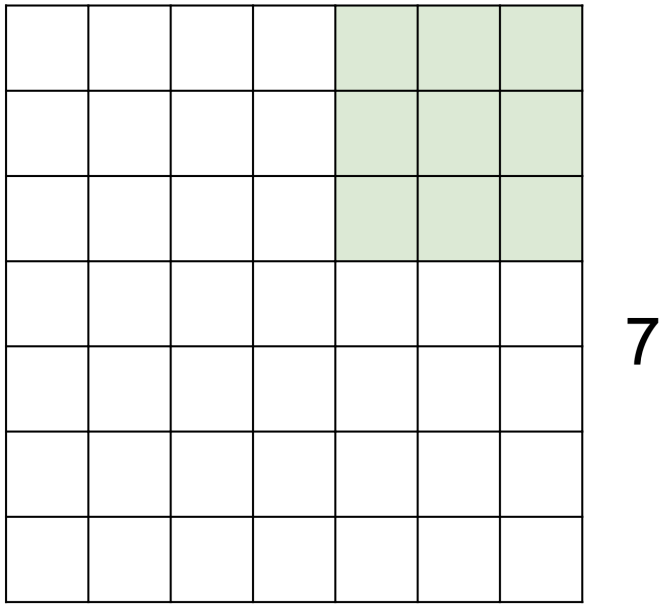
- A closer look at spatial dimensions

7×7 input
assume 3×3 filter



Convolutional layer spatial dimension analysis

- A closer look at spatial dimensions



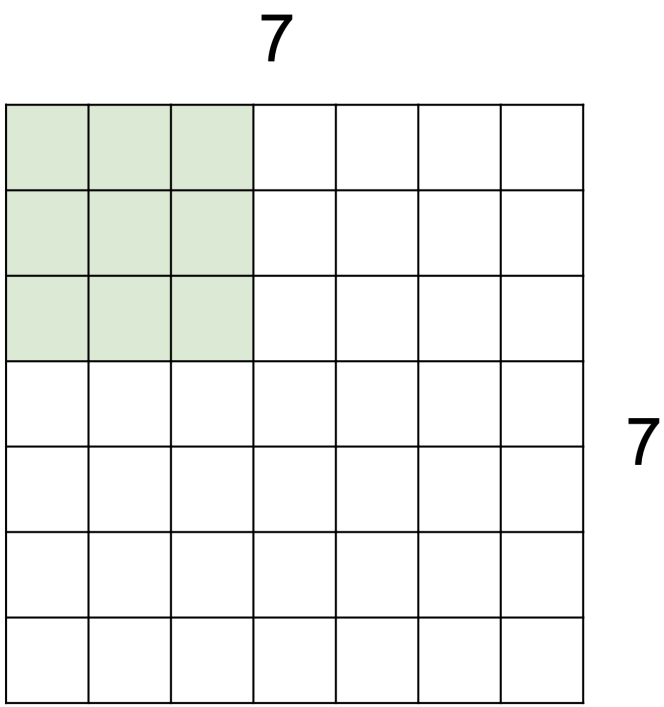
7×7 input

assume 3×3 filter

$\Longrightarrow$ 5×5 **output**

Convolutional layer spatial dimension analysis

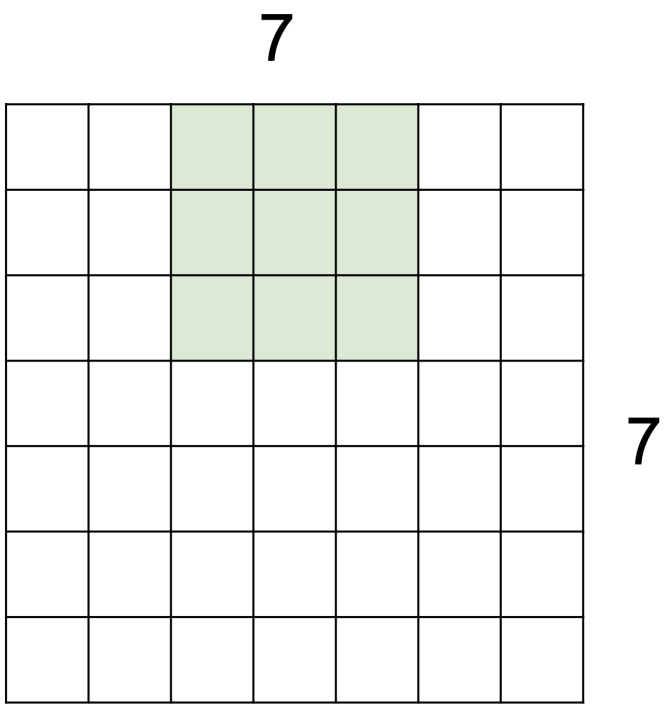
- A closer look at spatial dimensions



7×7 input
assume 3×3 filter
applied **with stride 2**

Convolutional layer spatial dimension analysis

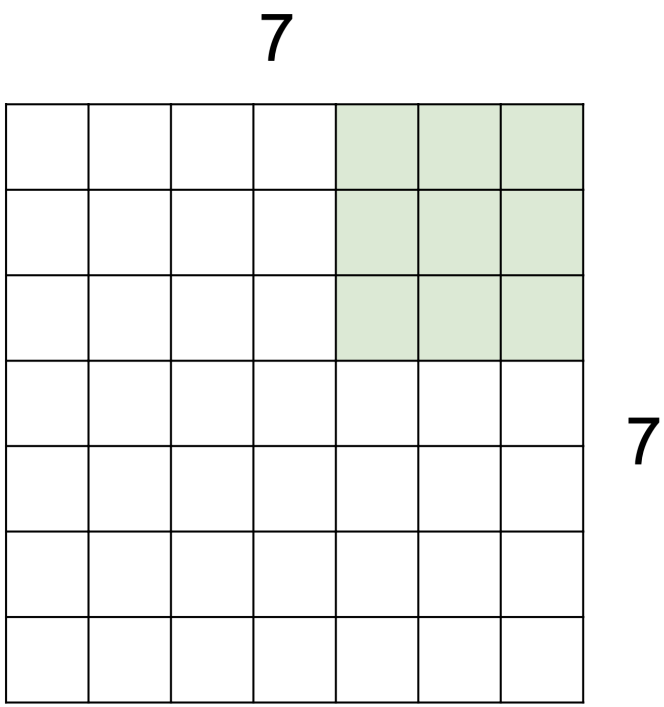
- A closer look at spatial dimensions



7×7 input
assume 3×3 filter
applied **with stride 2**

Convolutional layer spatial dimension analysis

- A closer look at spatial dimensions

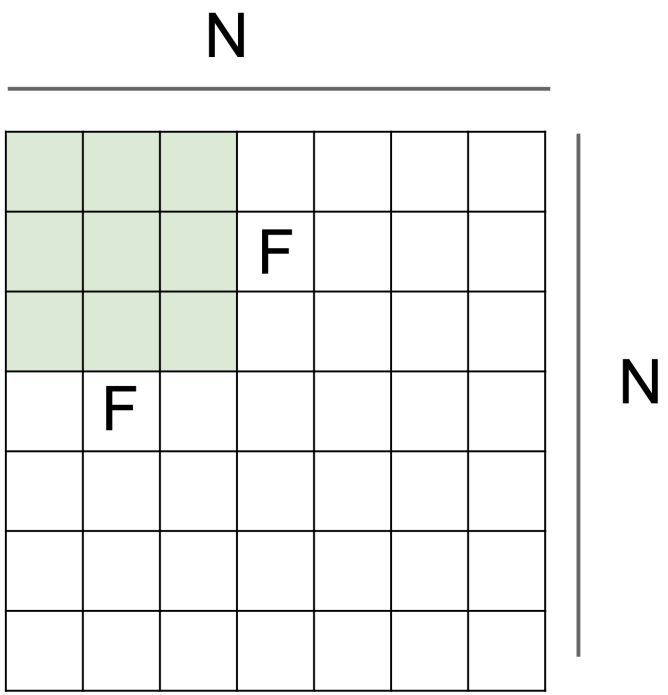


7×7 input
assume 3×3 filter
applied **with stride 2**

$\Longrightarrow$ 3×3 **output**

Convolutional layer spatial dimension analysis

- A closer look at spatial dimensions

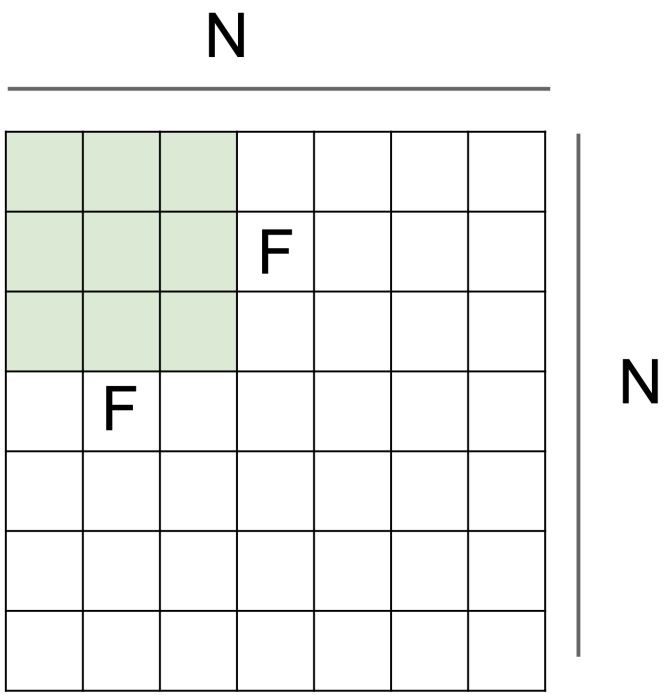


N: input dimension
F: filter size

Output size:
$(N - F)/stride + 1$

Convolutional layer spatial dimension analysis

- A closer look at spatial dimensions



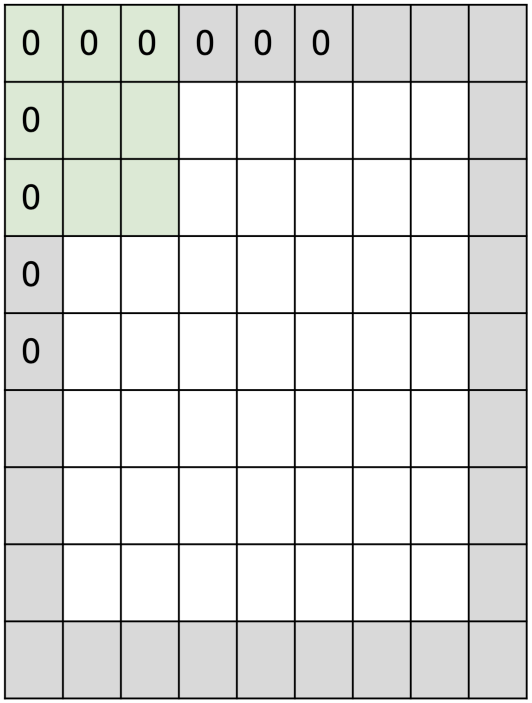
N: input dimension
F: filter size

Output size:
$(N - F)/stride + 1$

e.g. $N = 7, F = 3$:
stride 1 $\implies (7 - 3)/1 + 1 = 5$
stride 2 $\implies (7 - 3)/2 + 1 = 3$

Convolutional layer spatial dimension analysis

- In practice: common to zero pad the border.

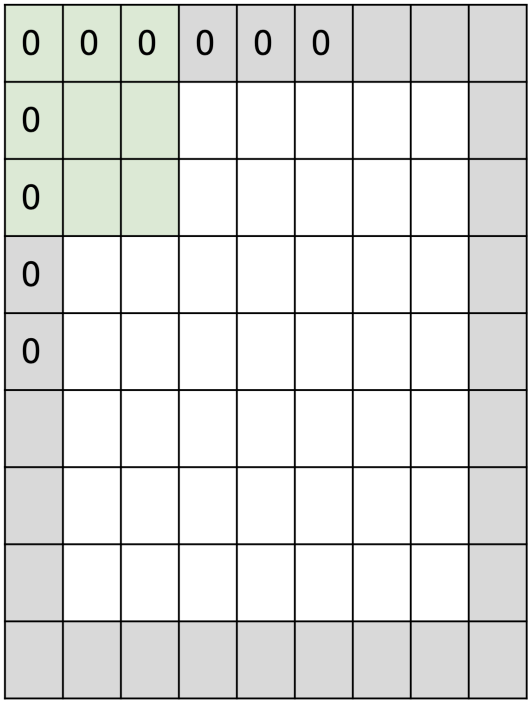


For example, input 7×7 image, use 3×3 filter, convolve with stride 1, padding with 1 pixel border

$\Longrightarrow$ what is the output size?

Convolutional layer spatial dimension analysis

- In practice: common to zero pad the border.



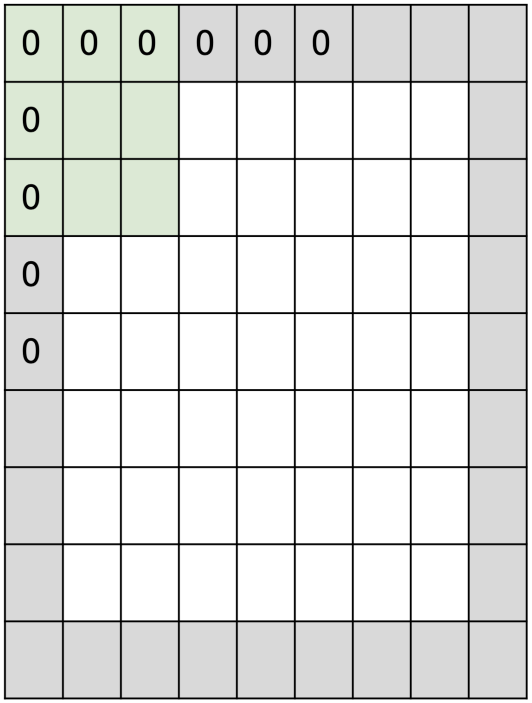
For example, input 7×7 image, use 3×3 filter, convolve with stride 1, padding with 1 pixel border

$\Longrightarrow$ what is the output size?

7×7 **output**

Convolutional layer spatial dimension analysis

- In practice: common to zero pad the border.



For example, input 7×7 image, use 3×3 filter, convolve with stride 1, padding with 1 pixel border

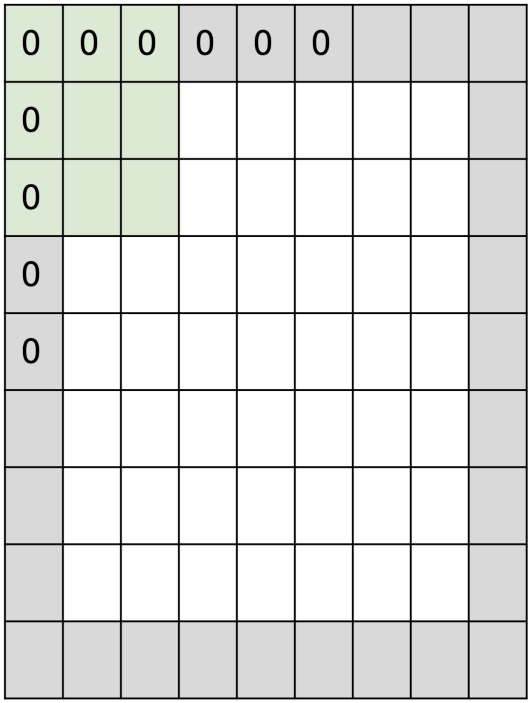
$\Longrightarrow$ what is the output size?

7×7 **output**

In practice, common to see convolutional layers using zero-padding with $(F-1)/2$, stride of 1, in order to preserve size spatially

Convolutional layer spatial dimension analysis

- In practice: common to zero pad the border.



For example, input 7×7 image, use 3×3 filter, convolve with stride 1, padding with 1 pixel border

$\Longrightarrow$ what is the output size?

7×7 **output**

In practice, common to see convolutional layers using zero-padding with $(F-1)/2$, stride of 1, in order to preserve size spatially

e.g. $F = 3 \implies$ zero pad with 1

$F = 5 \implies$ zero pad with 2

$F = 7 \implies$ zero pad with 3

Quiz

- For a convolutional layer:

  Input volume: $224 \times 224 \times 3$
  Use 50 8×8 filters with stride 2, pad 3

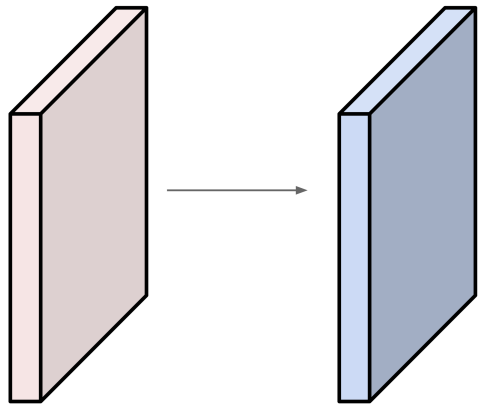
  What is the output feature map size?

  A $112 \times 112 \times 3$

  B $112 \times 112 \times 50$

  C $111 \times 111 \times 50$

  D $223 \times 223 \times 50$



Quiz

- For a convolutional layer:

  Input volume: $224 \times 224 \times 3$
  Use 50 8×8 filters with stride 2, pad 3

  What is the output feature map size?

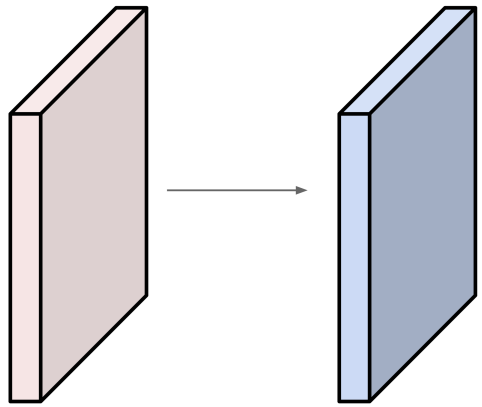
  A $112 \times 112 \times 3$

  B $112 \times 112 \times 50$

  C $111 \times 111 \times 50$

  D $223 \times 223 \times 50$

  Correct Answer: B
  $(224 + 2 \times 3 - 8)/2 + 1 = 112$
  spatially, so $112 \times 112 \times 50$



Quiz

- For a convolutional layer:

  Input volume: $224 \times 224 \times 3$
  50 8×8 filters with stride 2, pad 3

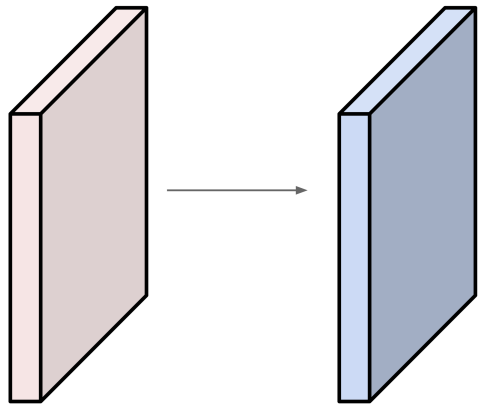
  Number of parameters in this layer?

  A $9,550$

  B $9,700$

  C $9,600$

  D $9,650$



Quiz

- For a convolutional layer:

  Input volume: $224 \times 224 \times 3$
  50 8×8 filters with stride 2, pad 3

  Number of parameters in this layer?

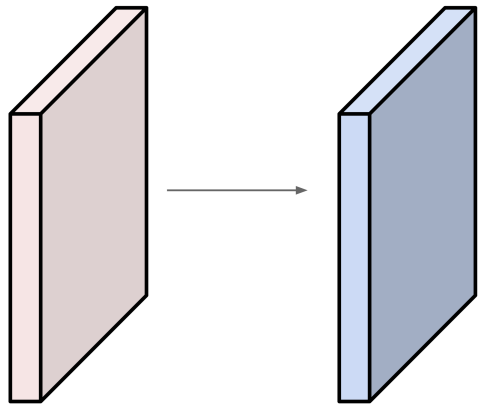
  A $9,550$

  B $9,700$

  C $9,600$

  D $9,650$

  Correct Answer: D
  each filter has $8 \times 8 \times 3 + 1 = 193$
  params $\implies$ $193 \times 50 = 9,650$
  ($+1$ for bias)



Convolutional layer – summary

- Receives a 3D volume of size $W_1 \times H_1 \times D_1$ and requires four hyper-parameters:
  - number of filters K
  - filter spatial extent F
  - the convolution stride S
  - the amount of zero padding P

Convolutional layer – summary

- Receives a 3D volume of size $W_1 \times H_1 \times D_1$ and requires four hyper-parameters:
  - number of filters K
  - filter spatial extent F
  - the convolution stride S
  - the amount of zero padding P
- Produces a volume of size $W_2 \times H_2 \times D_2$ with:
  - $W_2 = (W_1 + 2P - F)/S + 1$
  - $H_2 = (H_1 + 2P - F)/S + 1$
  - $D_2 = K$

Convolutional layer – summary

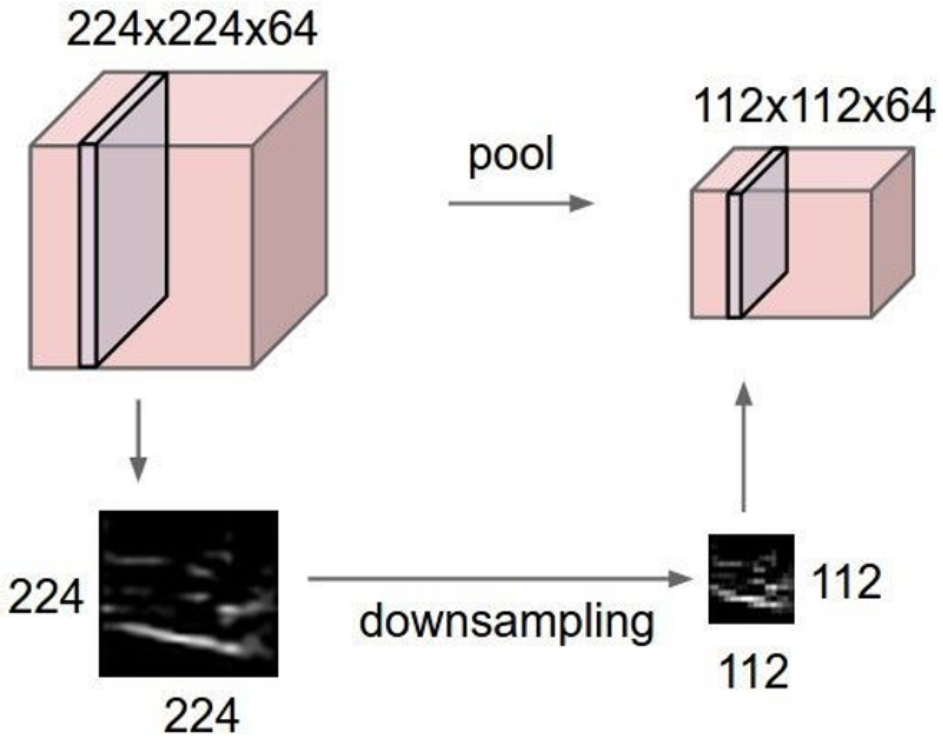
- Receives a 3D volume of size $W_1 \times H_1 \times D_1$ and requires four hyper-parameters:
  - number of filters K
  - filter spatial extent F
  - the convolution stride S
  - the amount of zero padding P
- Produces a volume of size $W_2 \times H_2 \times D_2$ with:
  - $W_2 = (W_1 + 2P - F)/S + 1$
  - $H_2 = (H_1 + 2P - F)/S + 1$
  - $D_2 = K$
- It introduces $F \times F \times D_1$ weights in each filter, in total, for a convolutional layer, it has $(F \times F \times D_1) \times K$ weights and K biases.

Convolutional layer – summary

- Receives a 3D volume of size $W_1 \times H_1 \times D_1$ and requires four hyper-parameters:
  - number of filters K
  - filter spatial extent F
  - the convolution stride S
  - the amount of zero padding P
- Produces a volume of size $W_2 \times H_2 \times D_2$ with:
  - $W_2 = (W_1 + 2P - F)/S + 1$
  - $H_2 = (H_1 + 2P - F)/S + 1$
  - $D_2 = K$
- It introduces $F \times F \times D_1$ weights in each filter, in total, for a convolutional layer, it has $(F \times F \times D_1) \times K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

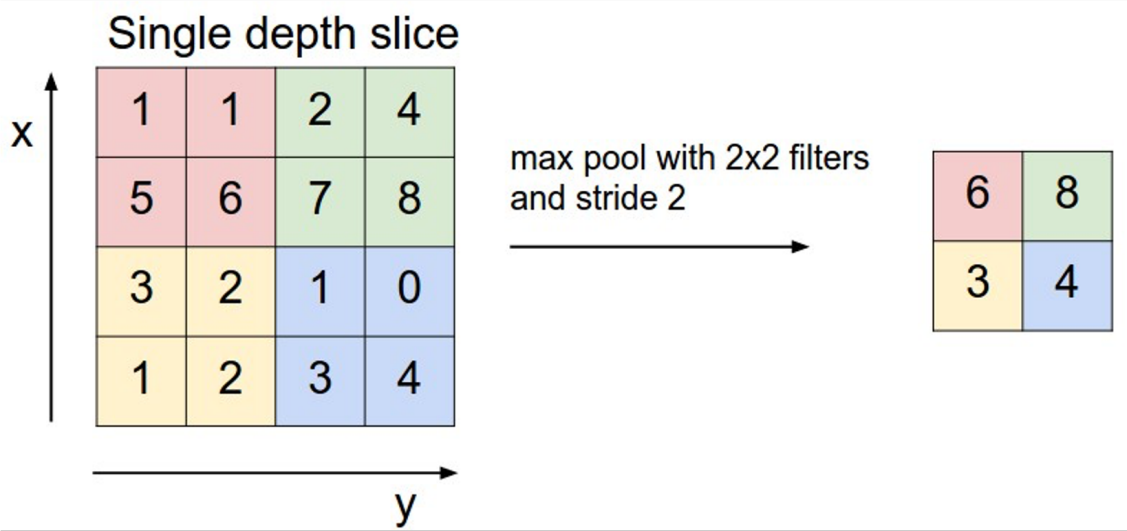
Pooling layer

- Make the representations smaller and more manageable
- Operate over each activation map independently



Pooling layer

- Max pooling layer example



- *Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Notice that the volume depth (i.e., feature channels) is preserved.*
- *In this example, the input volume of size* $[224 \times 224 \times 64]$ *is pooled with filter size* 2*, stride* 2 *into output volume of size* $[112 \times 112 \times 64]$.
- *The max pooling operation, here shown with a stride of* 2*. That is, each max is taken over* 4 *numbers (little* 2×2 *square).*

Pooling layer – summary

Let's assume input is $W_1 \times H_1 \times D$.

Pooling layer – summary

Let's assume input is $W_1 \times H_1 \times D$.

- Conv layer needs 2 hyperparameters:
  - pooling spatial extent F
  - the pooling stride S

Pooling layer – summary

Let's assume input is $W_1 \times H_1 \times D$.

- Conv layer needs 2 hyperparameters:
  - pooling spatial extent F
  - the pooling stride S
- This will produce an output of $W_2 \times H_2 \times D$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$

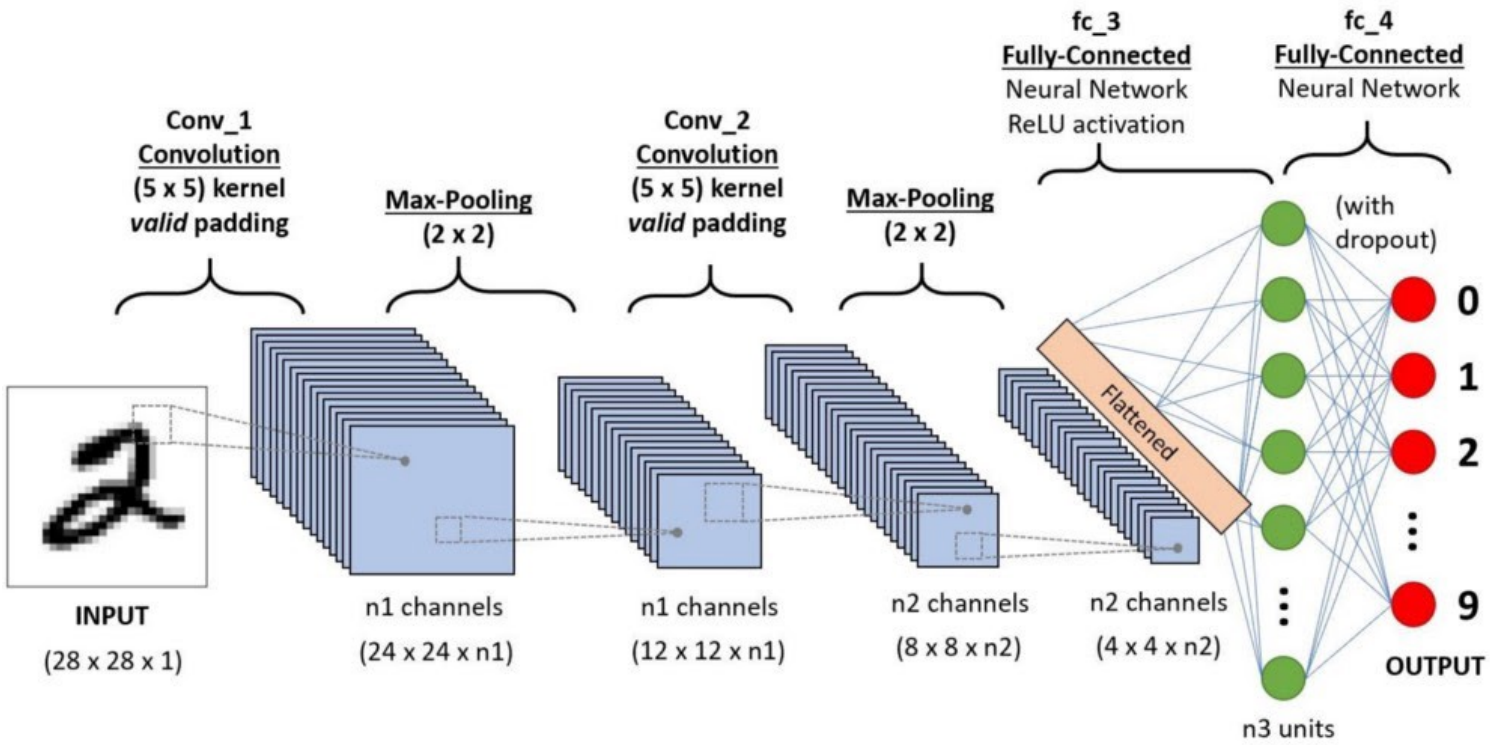
Pooling layer – summary

Let's assume input is $W_1 \times H_1 \times D$.

- Conv layer needs 2 hyperparameters:
  - pooling spatial extent F
  - the pooling stride S
- This will produce an output of $W_2 \times H_2 \times D$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
- Number of parameters: 0

Fully-connected layer

- Neurons in a fully-connected layer have full connections to all activations in the previous layer, as seen in regular Neural Networks.
- Their activations can hence be computed with a matrix multiplication followed by a bias offset.



Part 2. CNN for classification

CNN architectures

- Case studies on milestone CNN architectures
  - AlexNet
  - VGG
  - ResNet

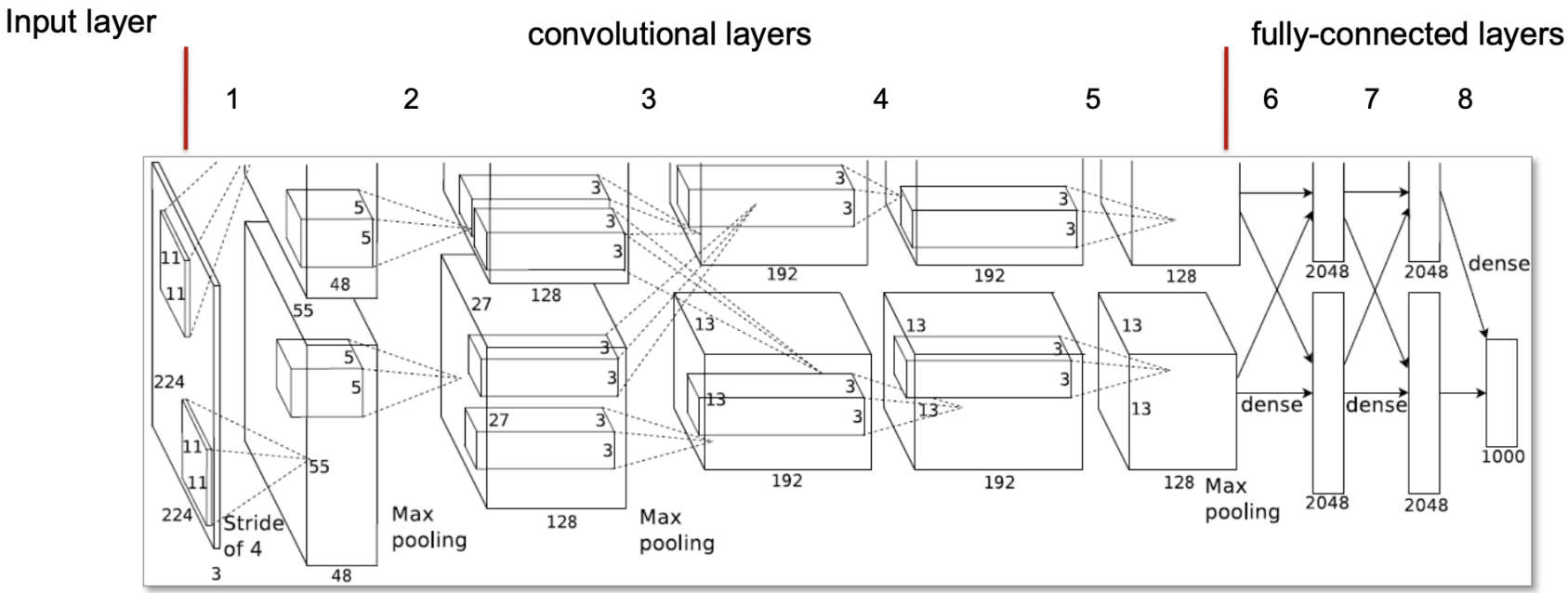
CNN architectures

- Case studies on milestone CNN architectures
  - AlexNet
  - VGG
  - ResNet
- Other well-known CNNs...
  - GoogLeNet
  - SENet
  - Wide ResNet
  - ResNeXt
  - DenseNet
  - MobileNet
  - NASNet
  - EfficientNet

  ...

AlexNet

- The first work that popularized Convolutional Networks in Computer Vision was the AlexNet, developed by Alex Krizhevsky, Ilya Sutskever and Geoff Hinton, which was the winner of ILSVRC 2012.
- Architecture:

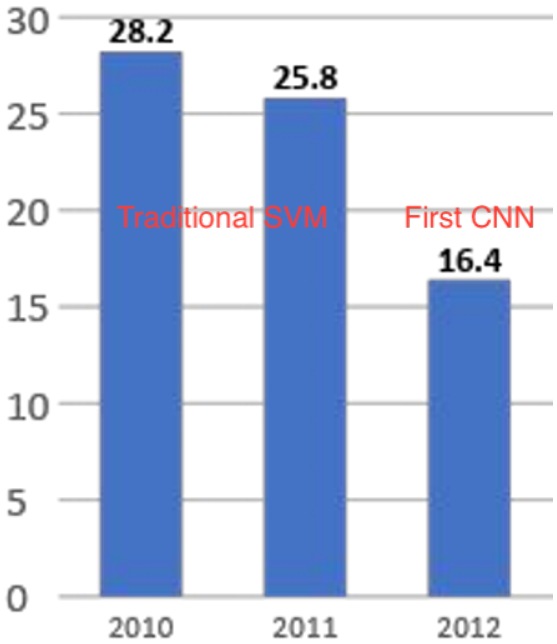


AlexNet

- Input: $227 \times 227 \times 3$ images
- Parameters:
  - $650,000$ neurons.
  - $60,000,000$ parameters.
  - $630,000,000$ connections.
  - Final feature layer: 4096-dimensional.

AlexNet

- Input: $227 \times 227 \times 3$ images
- Parameters:
  - $650,000$ neurons.
  - $60,000,000$ parameters.
  - $630,000,000$ connections.
  - Final feature layer: 4096-dimensional.



AlexNet

- Experimental results:

Model	Top-1	Top-5
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

Table 1: Comparison of results on ILSVRC-2010 test set. In *italics* are best results achieved by others.

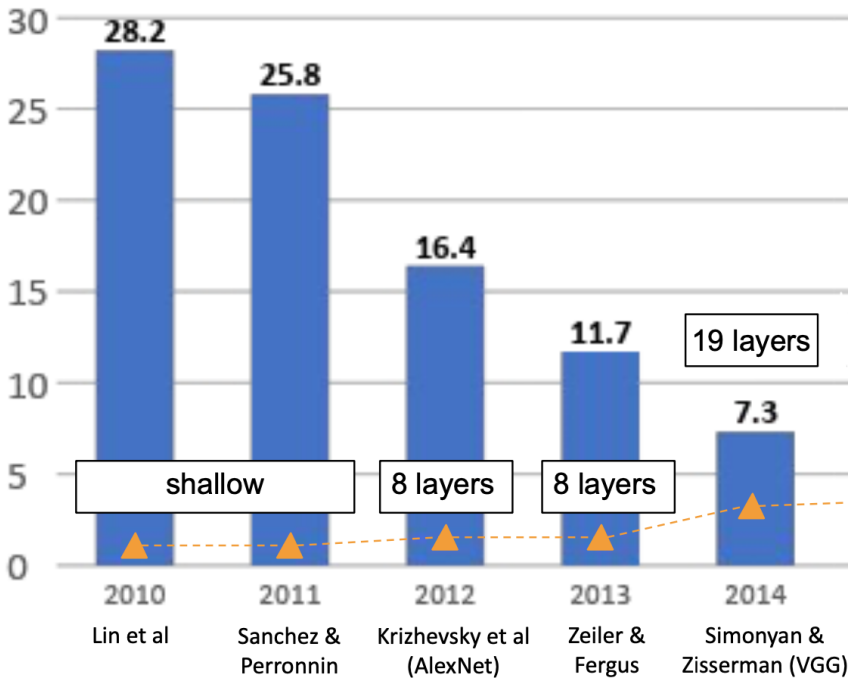
Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SIFT + FVs [7]	—	—	*26.2%*
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	**16.4%**
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	**15.3%**

Table 2: Comparison of error rates on ILSVRC-2012 validation and test sets. In *italics* are best results achieved by others. Models with an asterisk* were "pre-trained" to classify the entire ImageNet 2011 Fall release. See Section 6 for details.

Krizhevsky, A., Sutskever, I. and Hinton, G.E., Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems (NeurIPS), 2012.

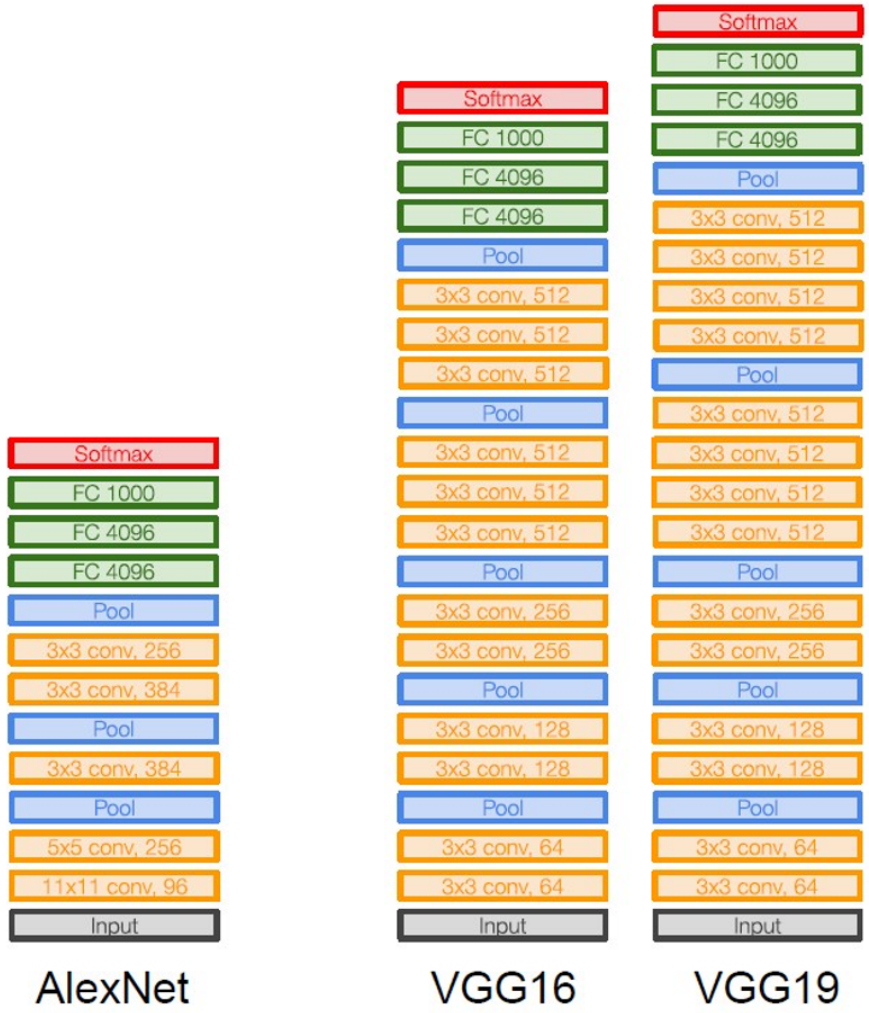
VGG

- The runner-up in ILSVRC 2014 was the network from Karen Simonyan and Andrew Zisserman.
- Main contributions:
  - Investigate the effect of CNN **depth** on recognition.
  - Use very **small** convolution filters through the net, significant improvement.



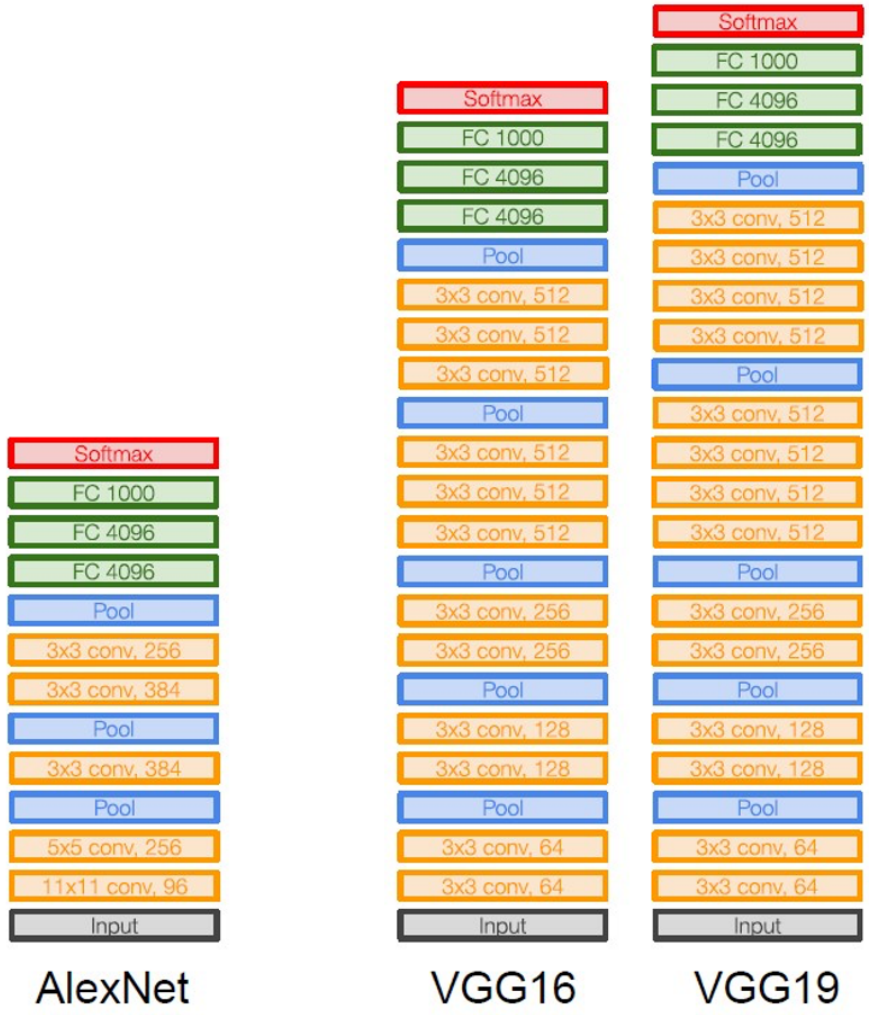
VGG

- Architecture:



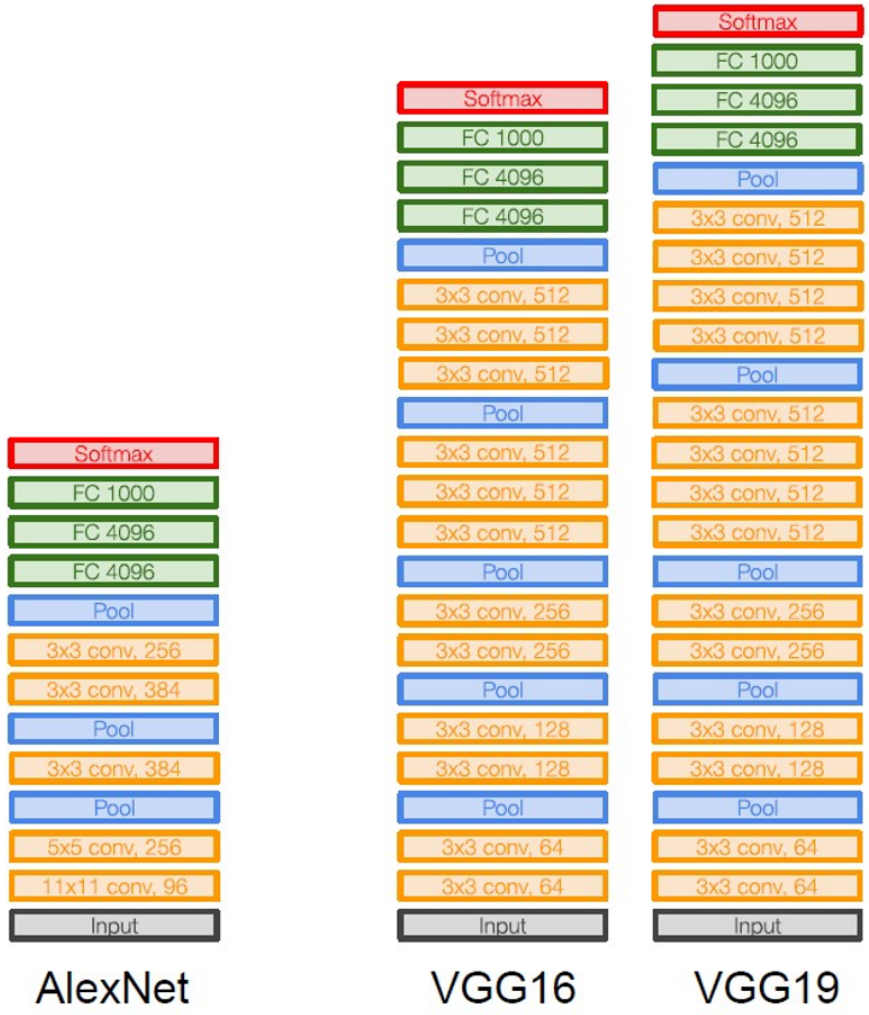
VGG

- Architecture:
  - Convolution kernels
    - 3×3: smallest size to capture the notion of left/right, up/down, center.
    - Padding: spatial resolution preservation.
    - Channels: from 64 to 512 with a factor of 2.



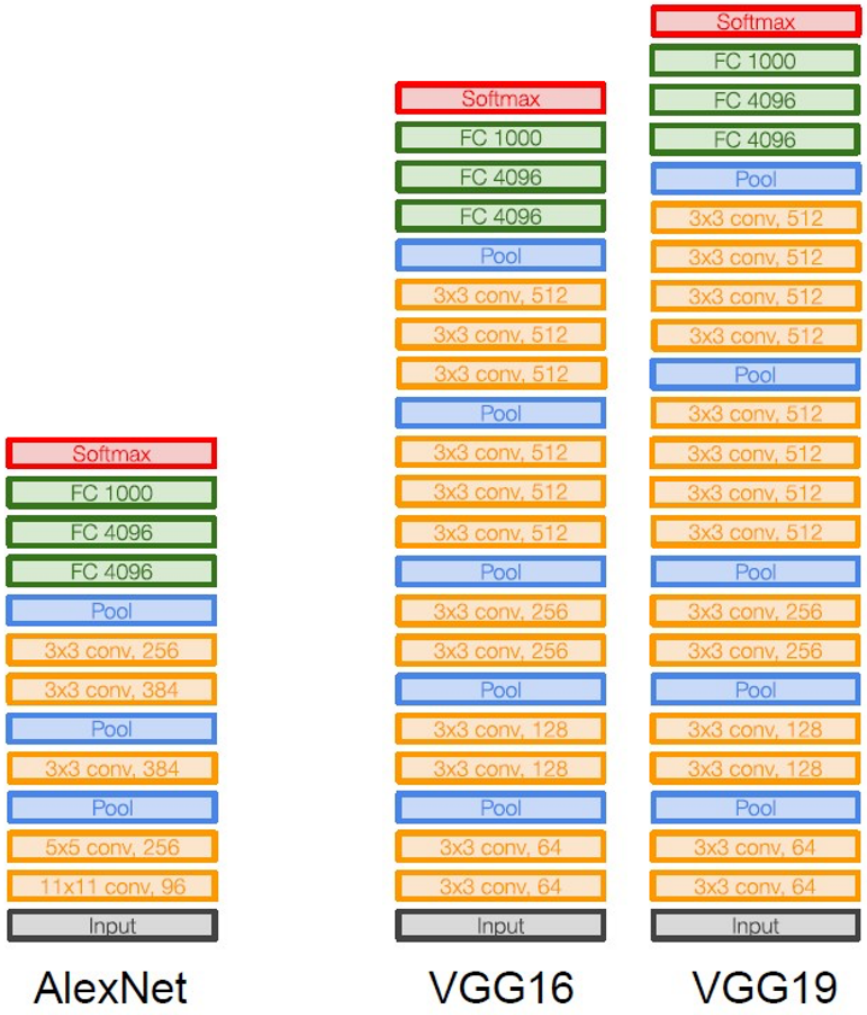
VGG

- Architecture:
  - Convolution kernels
    - 3×3: smallest size to capture the notion of left/right, up/down, center.
    - Padding: spatial resolution preservation.
    - Channels: from 64 to 512 with a factor of 2.
  - Max-pooling
    - 2×2 pixel window, with stride 2.



VGG

- Architecture:
  - Convolution kernels
    - 3×3: smallest size to capture the notion of left/right, up/down, center.
    - Padding: spatial resolution preservation.
    - Channels: from 64 to 512 with a factor of 2.
  - Max-pooling
    - 2×2 pixel window, with stride 2.
  - Non-linearity
    - ReLU



VGG

- Experimental results:

Table 7: **Comparison with the state of the art in ILSVRC classification**. Our method is denoted as "VGG". Only the results obtained without outside training data are reported.

Method	top-1 val. error (%)	top-5 val. error (%)	top-5 test error (%)
VGG (2 nets, multi-crop & dense eval.)	**23.7**	**6.8**	**6.8**
VGG (1 net, multi-crop & dense eval.)	24.4	7.1	7.0
VGG (ILSVRC submission, 7 nets, dense eval.)	24.7	7.5	7.3
GoogLeNet (Szegedy et al., 2014) (1 net)	-	7.9	
GoogLeNet (Szegedy et al., 2014) (7 nets)	-	**6.7**	
MSRA (He et al., 2014) (11 nets)	-	-	8.1
MSRA (He et al., 2014) (1 net)	27.9	9.1	9.1
Clarifai (Russakovsky et al., 2014) (multiple nets)	-	-	11.7
Clarifai (Russakovsky et al., 2014) (1 net)	-	-	12.5
Zeiler & Fergus (Zeiler & Fergus, 2013) (6 nets)	36.0	14.7	14.8
Zeiler & Fergus (Zeiler & Fergus, 2013) (1 net)	37.5	16.0	16.1
OverFeat (Sermanet et al., 2014) (7 nets)	34.0	13.2	13.6
OverFeat (Sermanet et al., 2014) (1 net)	35.7	14.2	-
Krizhevsky et al. (Krizhevsky et al., 2012) (5 nets)	38.1	16.4	16.4
Krizhevsky et al. (Krizhevsky et al., 2012) (1 net)	40.7	18.2	-

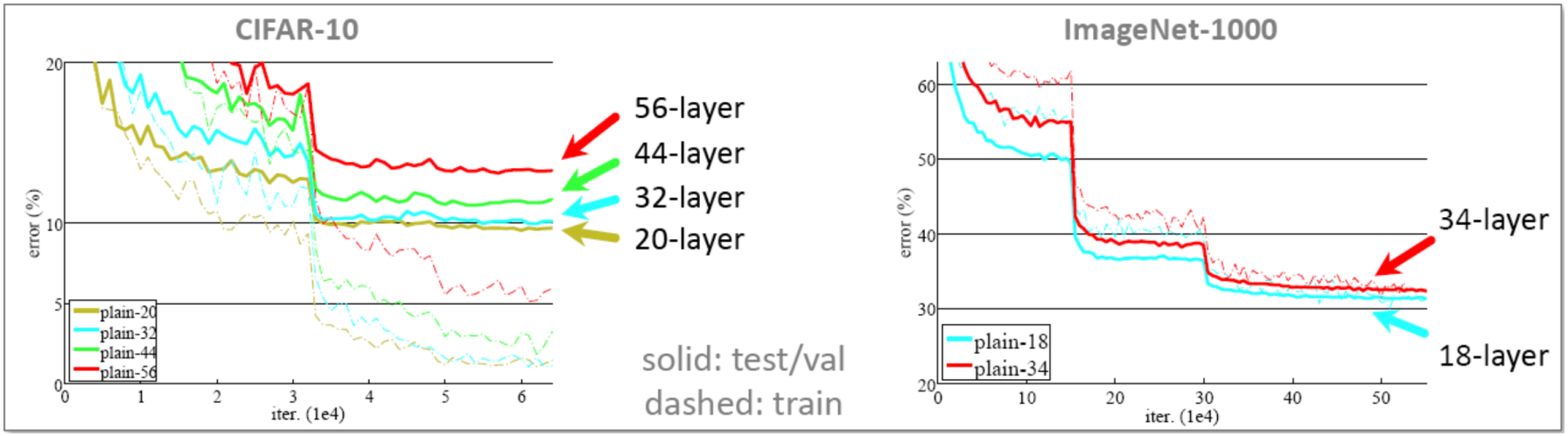
Simonyan, K. and Zisserman, A., 2014. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

ResNet

From AlexNet to VGG, it is obvious that ***deeper network*** *with more layers will perform better, what if we simply stack more layers?*

ResNet

*From AlexNet to VGG, it is obvious that **deeper network** with more layers will perform better, what if we simply stack more layers?*

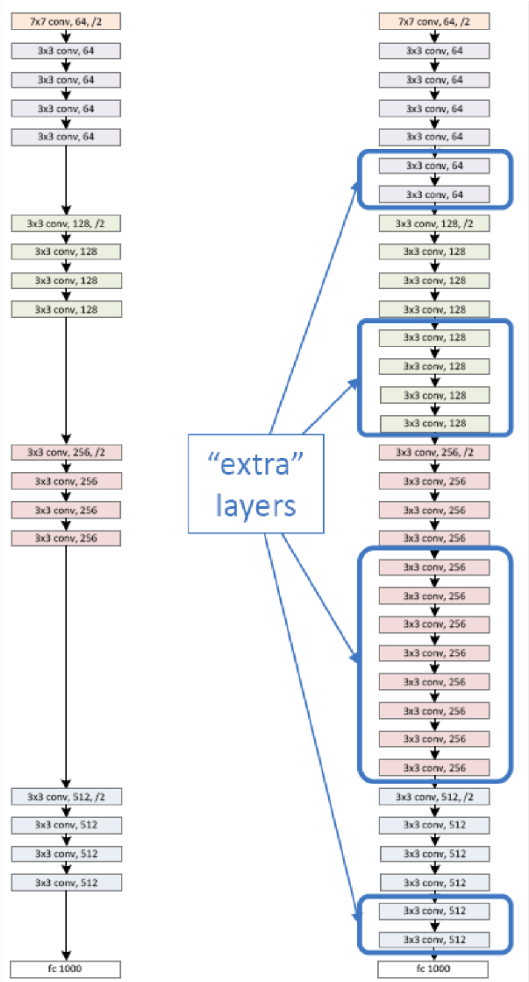


- "Overly deep" plain nets have higher training error.
- A general phenomenon, observed in many datasets.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". CVPR 2016. (Best Paper Award)

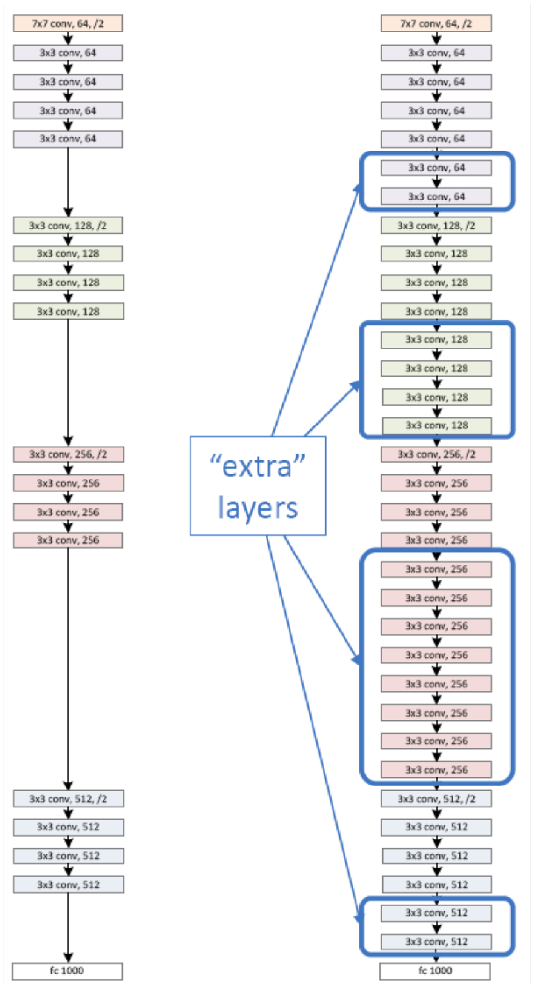
ResNet

- A deeper model should not have higher training error.



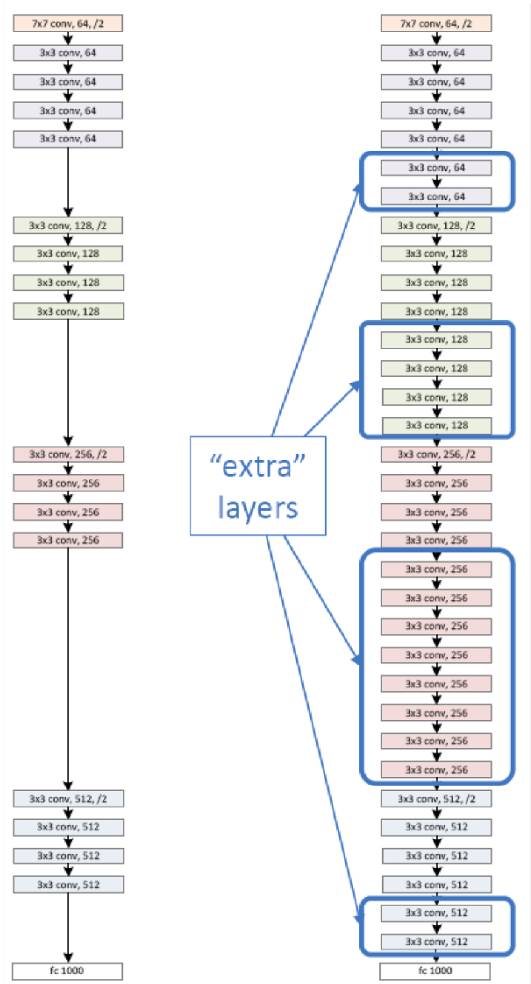
ResNet

- A deeper model should not have higher training error.
- A solution by construction:
  - original layers: copied from a learned shallower model.
  - extra layers: set as identity, at least the same training error.



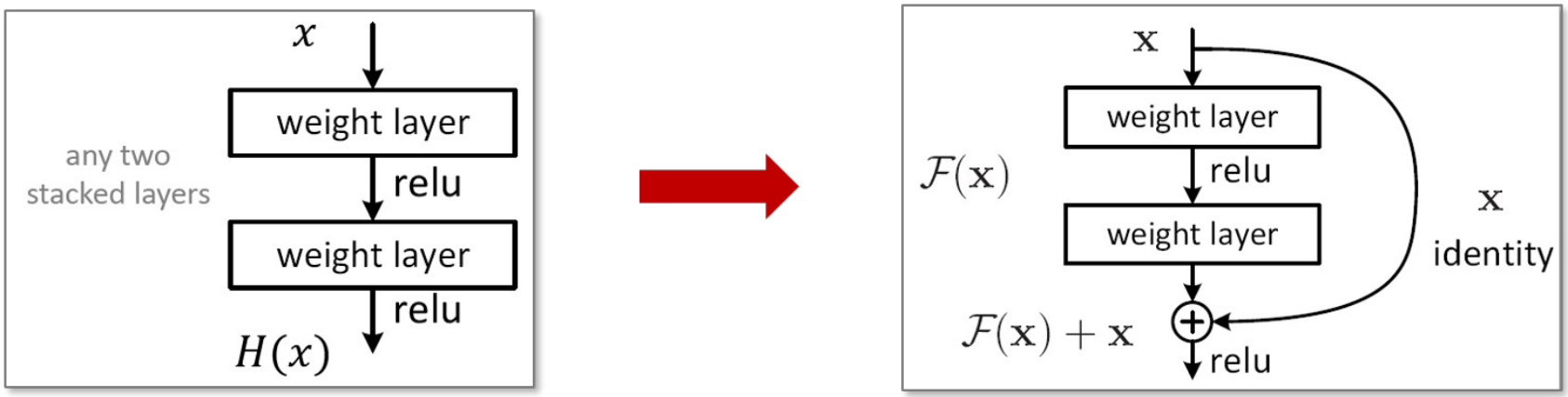
ResNet

- A deeper model should not have higher training error.
- A solution by construction:
  - original layers: copied from a learned shallower model.
  - extra layers: set as identity, at least the same training error.
- Optimization difficulties: solvers cannot find the solution when going deeper, it unlikely to be caused by vanishing gradients.

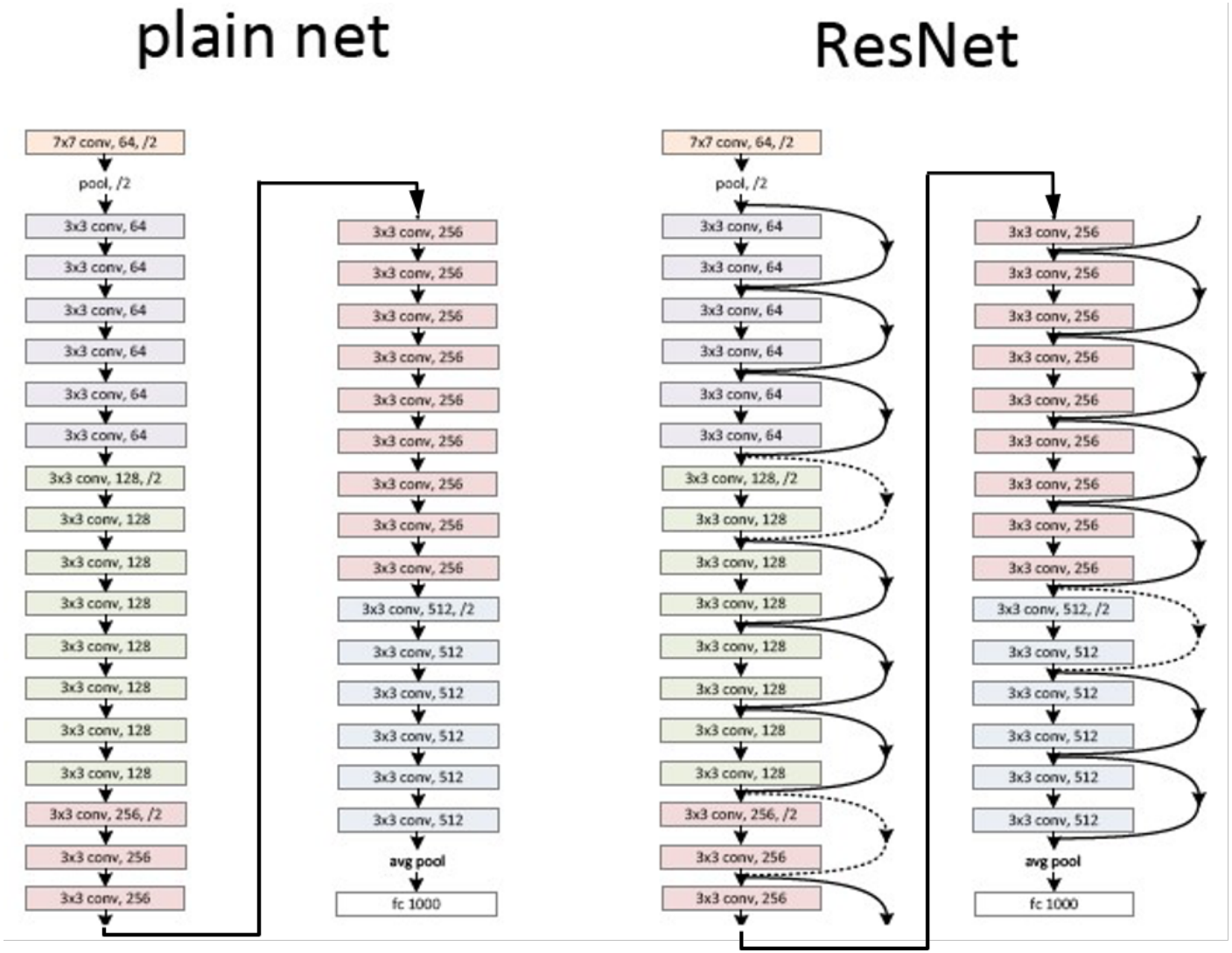


ResNet

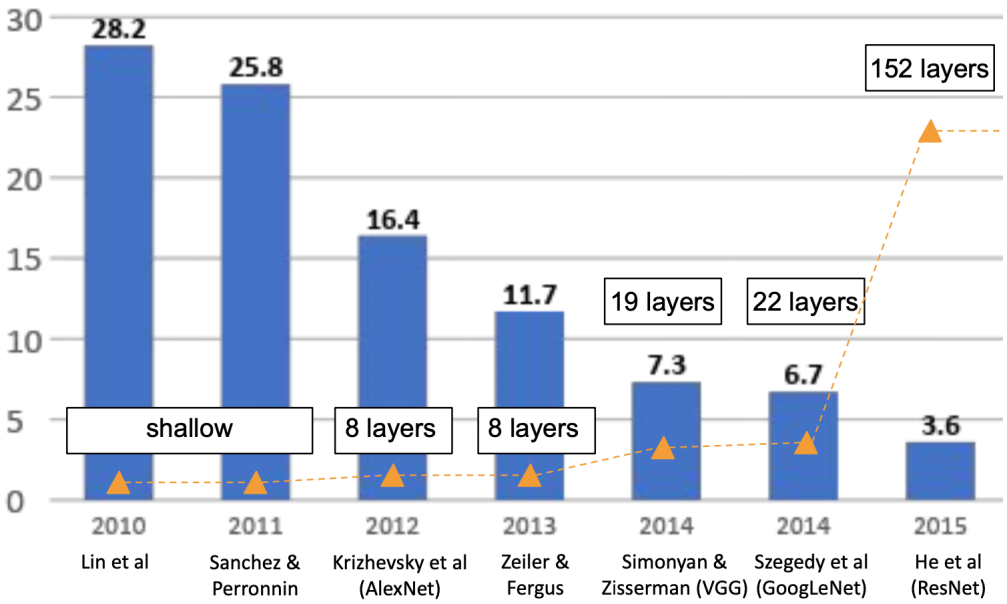
- Building block.
  - Plain net: $H(x)$ is any desired mapping, hope the 2 weight layers fit $H(x)$.
  - Residual net: $H(x)$ is any desired mapping, hope the 2 weight layers fit $F(x)$ and let $H(x) = F(x) + x$.
- F(x) is a residual mapping, which is much easier to optimize.



ResNet


plain net
ResNet
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
avg pool
fc 1000

ResNet



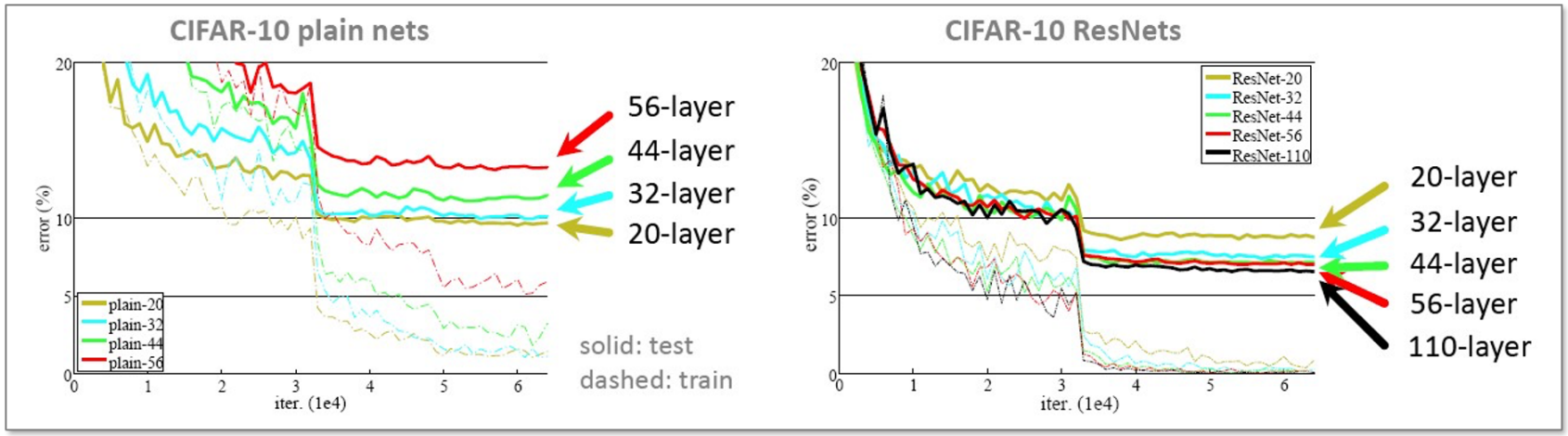
MSRA @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**
  - ImageNet Classification: "*Ultra-deep*" (quote Yann) 152-layer nets
  - ImageNet Detection: 16% better than 2nd
  - ImageNet Localization: 27% better than 2nd
  - COCO Detection: 11% better than 2nd
  - COCO Segmentation: 12% better than 2nd

Beat "human performance" of 5.1%! (Russakovsky 2014)

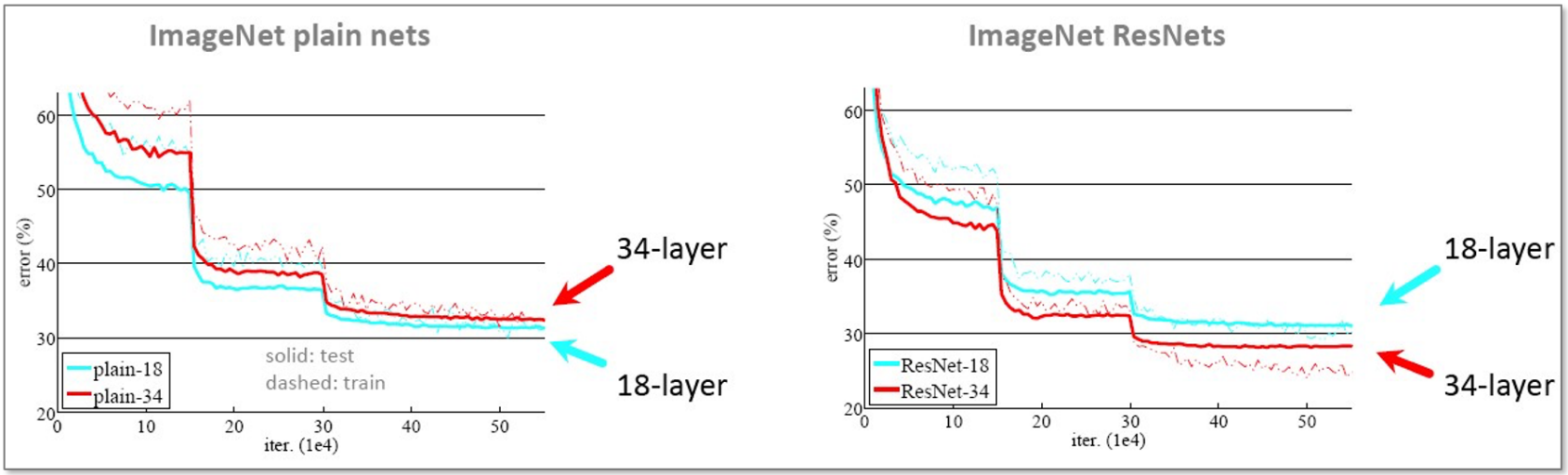
ResNet

- Experimental results: CIFAR10
  - Deep ResNets can be trained without difficulties.
  - Deeper ResNets have lower training error, and also lower test error.



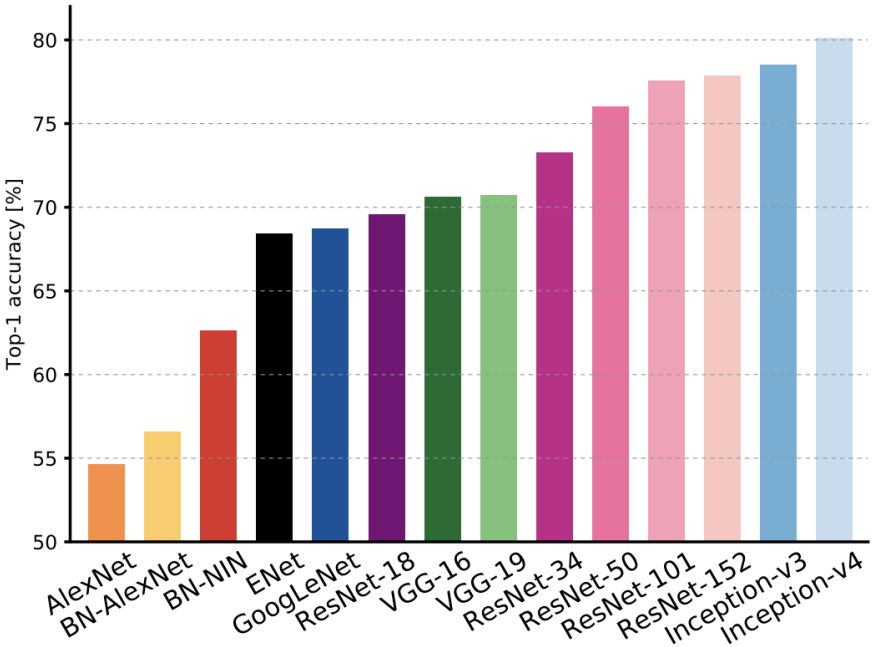
ResNet

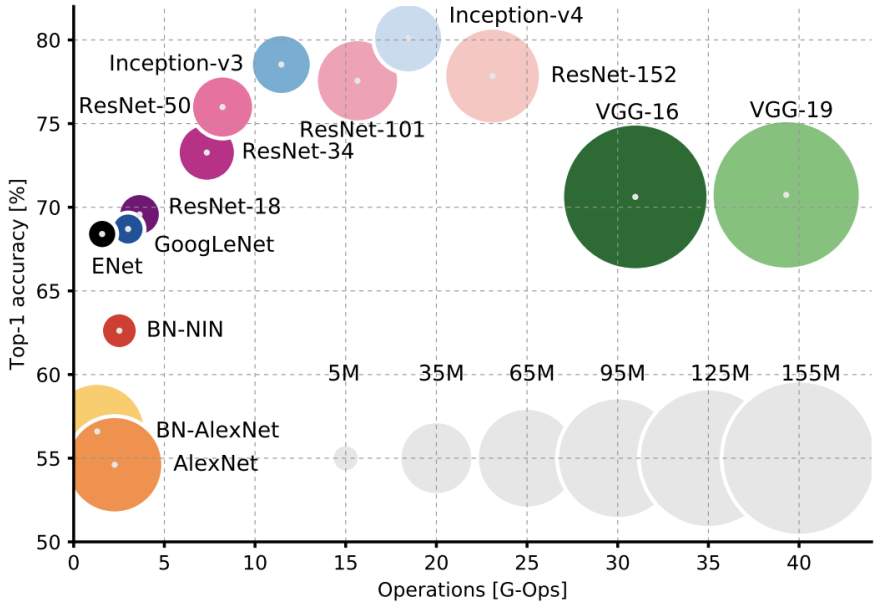
- Experimental results: ImageNet
  - Deep ResNets can be trained without difficulties.
  - Deeper ResNets have lower training error, also lower test error.



Comparing complexity

- Different CNN models' accuracy and computation cost



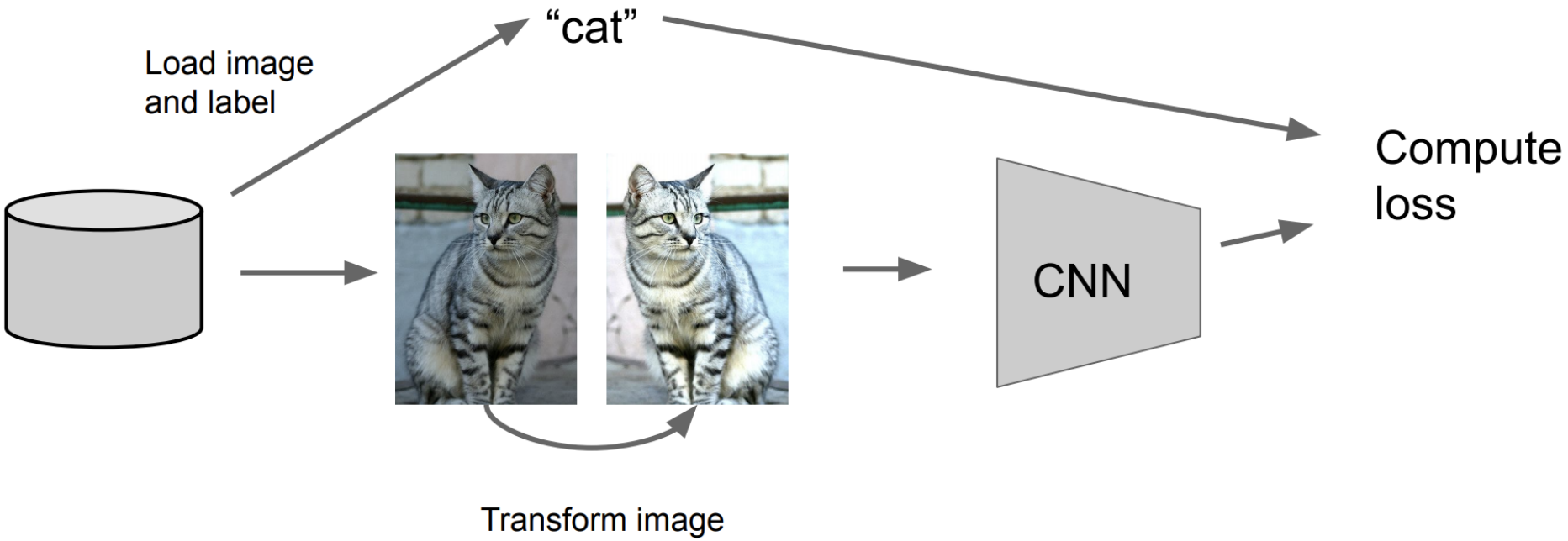


Canziani, A., Paszke, A. and Culurciello, E., 2016. An analysis of deep neural network models for practical applications. arXiv preprint arXiv:1605.07678.

Part 3. Tricks for CNN training

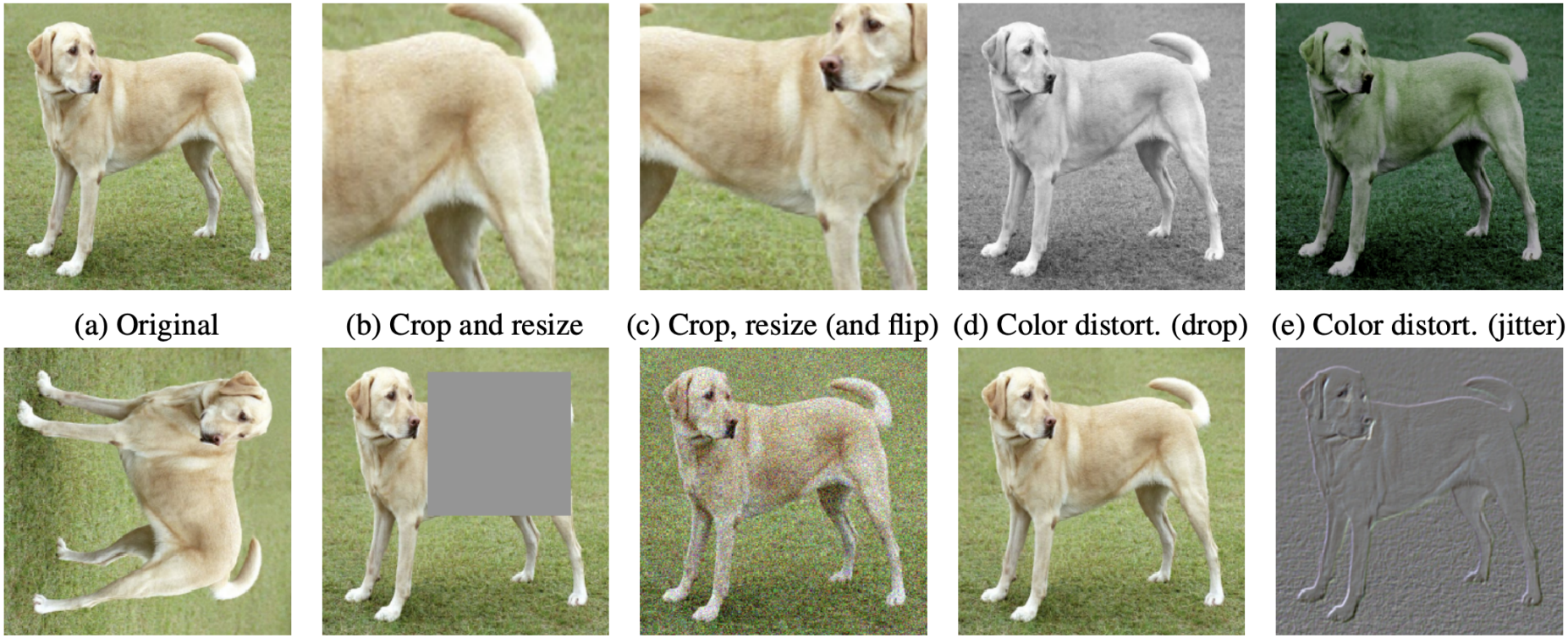
Data augmentation

- Data augmentation for images at the input layer



Data augmentation

- Some examples



(a) Original (b) Crop and resize (c) Crop, resize (and flip) (d) Color distort. (drop) (e) Color distort. (jitter)

(f) Rotate $\{90°, 180°, 270°\}$ (g) Cutout (h) Gaussian noise (i) Gaussian blur (j) Sobel filtering

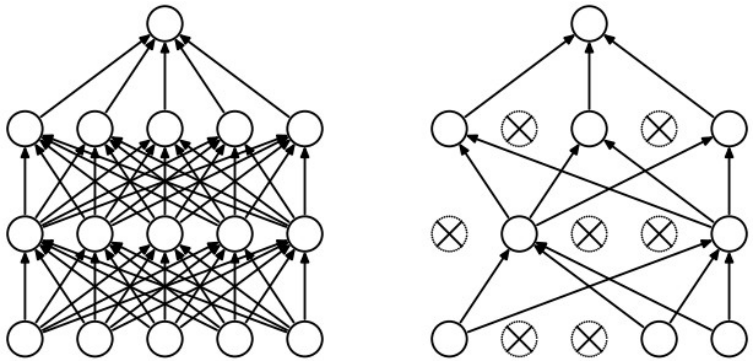
Chen, T., Kornblith, S., Norouzi, M. and Hinton, G., 2020, November. A simple framework for contrastive learning of visual representations. In International conference on machine learning (ICML), 2020.

Data augmentation - benefits

- Adding more training data into the models.
- Alleviating data scarcity for learning better models.
- Reducing data overfitting (i.e., the learned network corresponds too closely to a limited set of data points). Augmentations help create variability in data.
- Helping resolve class imbalance issues in classification.
- Reducing costs of collecting and labeling data.
- Increasing generalization ability of the models.

Dropout

- Dropout is an extremely effective yet simple regularization technique by Srivastava et al.
- While training, dropout is implemented by only keeping a neuron active with some probability p (a hyperparameter, usually $p = 0.5$)
- This means that a neuron is temporally removed in the forward pass and any weight updates are not applied to this neuron on the backward pass.



(a) Standard Neural Net (b) After applying dropout.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 2014.

Dropout - benefits

- Preventing all neurons in a layer from synchronously optimizing their weights. Random dropout prevents all the neurons from converging to the same goal (or called co-adaptation).
- In other words, if neurons are randomly "turned-off" during network training, the remaining neurons will have to step in and handle the representation required to make predictions for the missing neurons.

Dropout - benefits

- Preventing all neurons in a layer from synchronously optimizing their weights. Random dropout prevents all the neurons from converging to the same goal (or called co-adaptation).
- In other words, if neurons are randomly "turned-off" during network training, the remaining neurons will have to step in and handle the representation required to make predictions for the missing neurons.
- Dropout prevents co-adaptation by making the presence of other neurons unreliable. Thus, a neuron cannot rely on other specific neurons to make predictions. Instead, it must perform well in a wide variety of different contexts provided by other neurons. Therefore, dropout improves model generalization capability.
- Dropout, in practice, as a regularization of the deep neural networks to reduce network overfitting.

Part 4. Summary

Summary

- Basic concepts
  - Convolutional layer
  - Pooling layer
  - Fully-connected layer
- CNN architectures
  - AlexNet
  - VGG
  - ResNet
- Training CNNs
  - Data augmentation
  - Dropout

Reading materials

- Stanford CS231n: Convolutional Neural Networks for Visual Recognition http://cs231n.stanford.edu/
- Chapter 9 - Convolutional Networks, Deep learning. Goodfellow, I., Bengio, Y. and Courville, A., MIT press, 2016.

ESTR3108

Invited talk